49. Physical Coding Sublayer (PCS) for 64B/66B, type 10GBASE-R

49.1 Overview

49.1.1 Scope

This clause specifies the Physical Coding Sublayer (PCS) that is used in the family of 10GBASE-R Physical Layer devices (PHYs): 10GBASE-SR, 10GBASE-LR, 10GBASE-ER, 10GBASE-LRM, and 10GBASE-KR. Alternatively, this PCS can connect to a WAN Interface Sublayer (WIS), which produces the 10GBASE-W encoding (10GBASE-R encoded data stream encapsulated into frames compatible with SONET and SDH networks) for transport by the 10GBASE-W Physical Layer devices: 10GBASE-SW, 10GBASE-LW, and 10GBASE-EW. The term 10GBASE-R is used when referring generally to Physical Layers using the PCS defined here.

The 10GBASE-R is based on a 64B/66B code. The 64B/66B code supports data and control characters, while maintaining robust error detection.

10GBASE-R PCS maps the interface characteristics of the WIS when present, and the PMA sublayer to the services expected by the Reconciliation and XGXS sublayers. 10GBASE-R can be extended to support any full duplex medium requiring only that the medium be compliant at the PMA level. 10GBASE-R PCS may be attached through the PMA sublayer to a LAN PMD sublayer supporting a data rate of 10 Gb/s or it may be attached to a WAN PMD through the WIS and PMA sublayers. When attached to a WAN sublayer, this PCS adapts the data stream to the WAN data rate.

49.1.2 Objectives

The following are the objectives of 10GBASE-R:

a) Support the full duplex Ethernet MAC.
b) Provide 10 Gb/s data rate at the XGMII.
c) Support LAN PMDs operating at 10 Gb/s and WAN PMDs operating at SONET STS-192c/SDH VC-4-64c rate.
d) Support cable plants using cabled optical fiber compliant with ISO/IEC 11801:1995 as specified in Clause 52 and Clause 68.
e) Allow for a nominal network extent of up to 40 km.
f) Support a BER objective of 10^{-12}.

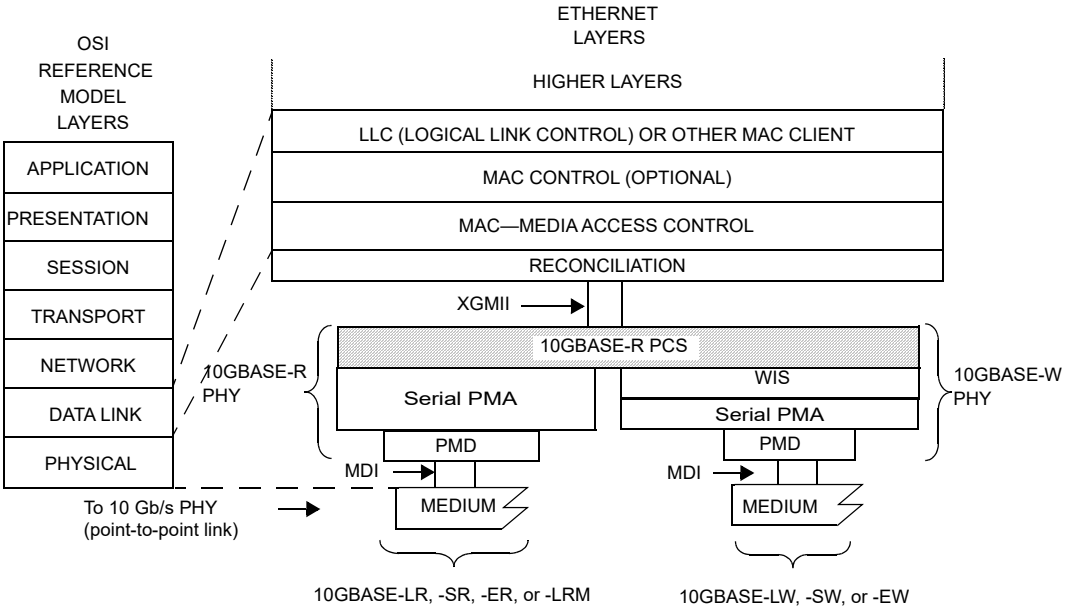
49.1.3 Relationship of 10GBASE-R to other standards

Figure 49–1 depicts the relationships among the 10GBASE-R sublayers (shown shaded), the Ethernet MAC and reconciliation layers, and the higher layers.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.1.4 Summary of 10GBASE-R and 10GBASE-W sublayers

The following subclauses provide an overview of the 10GBASE-R and 10GBASE-W sublayers.[92] Figure 49–1 depicts the relationship between the 10GBASE-R PCS and its associated sublayers.



MDI = MEDIUM DEPENDENT INTERFACE
PCS = PHYSICAL CODING SUBLAYER
PHY = PHYSICAL LAYER DEVICE
PMA = PHYSICAL MEDIUM ATTACHMENT
PMD = PHYSICAL MEDIUM DEPENDENT
WIS = WAN INTERFACE SUBLAYER
XGMII = 10 GIGABIT MEDIA INDEPENDENT INTERFACE

NOTE—The PMD sublayers are mutually independent.

PMD TYPES:
Medium:
E = PMD FOR FIBER—1550 nm WAVELENGTH
L = PMD FOR FIBER—1310 nm WAVELENGTH
S = PMD FOR FIBER—850 nm WAVELENGTH
M = PMD WITH DISPERSION COMPENSATION FOR MULTI-MODE FIBER
Encoding:
R = 64B/66B ENCODED WITHOUT WIS
W = 64B/66B ENCODED WITH WIS

Figure 49–1—10GBASE-R PCS relationship to the ISO/IEC Open Systems Interconnection (OSI) reference model and IEEE 802.3 Ethernet model

49.1.4.1 Physical Coding Sublayer (PCS)

The PCS service interface is the 10 Gigabit Media Independent Interface (XGMII), which is defined in Clause 46. The XGMII provides a uniform interface to the Reconciliation Sublayer for all 10 Gb/s PHY implementations (e.g., not only 10GBASE-R but also other types of 10 Gigabit PHY entities).

The 10GBASE-R PCS provides all services required by the XGMII, including the following:

a) Encoding (decoding) of eight XGMII data octets to (from) 66-bit blocks (64B/66B).
b) Transferring encoded data to (from) the PMA in 16 bit transfers.
c) When connected to a WAN PMD, deleting (inserting) idles to compensate for the rate difference between the MAC and PMD.

[92] The 10GBASE-R PHY consists of that portion of the Physical Layer between the MDI and XGMII consisting of the PCS, PMA, and PMD sublayers. The 10GBASE-R PHY is roughly analogous to the 100BASE-X PHY.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

d) Determining when a functional link has been established and informing the management entity via the MDIO when the PHY is ready for use.

49.1.4.2 WAN Interface Sublayer (WIS)

The WIS provides a medium-independent means for the PCS to operate over WAN links. It creates a 10GBASE-W encoding by encapsulating the encoded data steam from the 10GBASE-R PCS in frames compatible with SONET and SDH transmission formats. The WIS is specified in Clause 50.

49.1.4.3 Physical Medium Attachment (PMA) sublayer

The PMA provides a medium-independent means for the PCS to support the use of a range of physical media. The 10GBASE-R PMA performs the following functions:

a) Mapping of transmit and receive data streams between the PCS or WIS and PMA via the PMA service interface.
b) Serialization (deserialization) of bits for transmission (reception) on the underlying serial PMD.
c) Recovery of clock from the received data stream.
d) Mapping of transmit and receive bits between the PMA and PMD via the PMD service interface.
e) Optionally provides data loopback at the PMA service interface.

The PMA is specified in Clause 51.

49.1.4.4 Physical Medium Dependent (PMD) sublayer

The PMD and its media are specified in Clause 52 and Clause 68.

The MDI, logically subsumed within each PMD subclause, is the actual medium attachment for the various supported media.

49.1.4.5 Bit ordering across 10GBASE-R and 10GBASE-W sublayers.

The ordering of bits and octets for the case when no WIS is present between the PCS and PMA is shown in Figure 49–2. The diagram depicts the bit mappings for the data path at the service interfaces.

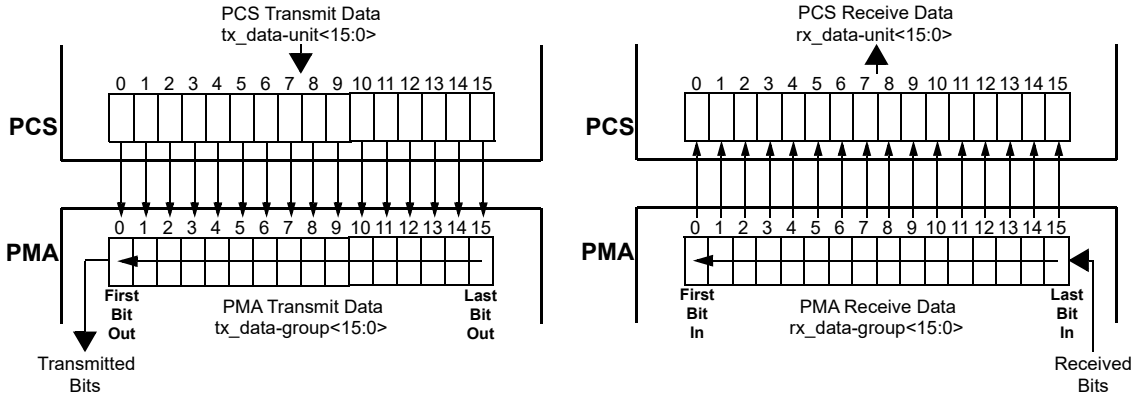


Figure 49–2—10GBASE-R PHY transmission order

The bit/octet ordering when a WIS is interposed between the PCS and PMA is shown in Figure 49–3. The details of WIS frame generation, as well as internal interfaces, have been omitted. Note that since SONET

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

convention is to send the most significant bit (MSB) of each octet first and Ethernet convention is to send the least significant bit (LSB) of each octet first, the LSBs from the PCS octets are mapped into the MSBs of the WIS octets. It should be clear from the diagram that the Ethernet order of transmission (i.e., least-significant bit first), and hence the error detecting properties of the Ethernet CRC, are preserved in spite of the different bit ordering conventions followed by the WIS. Also, the SONET/SDH bit labeling conventions are different from the usual IEEE 802.3 bit labeling. The bits of a SONET/SDH octet are labeled from 1 to 8 with bit 1 being the MSB. Ethernet conventions label bits of an n-bit field from 0 to n–1 with bit 0 being the LSB. Figure 49–3 shows the results of these conventions. For example, tx_data-unit<0> through tx_data-unit<7> map to bits 1 through 8 respectively of a WIS octet.

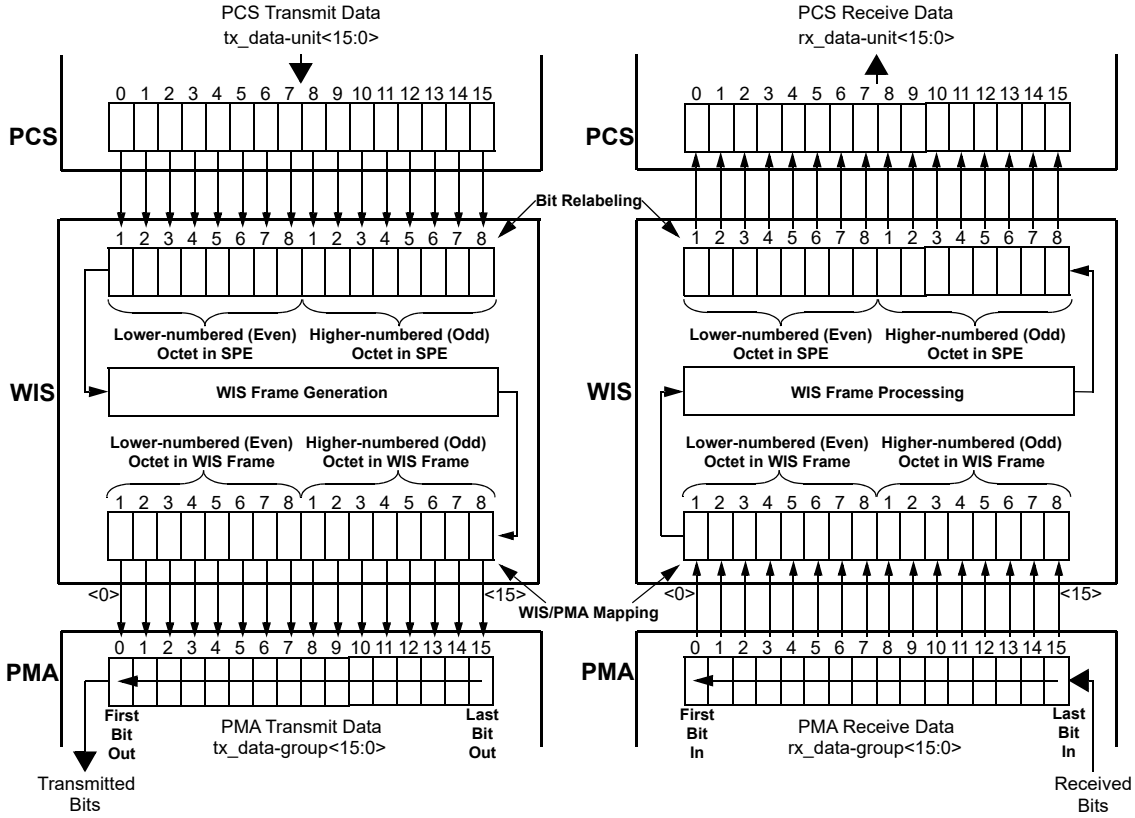


Figure 49–3—10GBASE-W PHY Transmission order

49.1.5 Inter-sublayer interfaces

There are a number of interfaces employed by 10GBASE-R. Some (such as the PMA service interface) use an abstract service model to define the operation of the interface. The PCS service interface is the XGMII that is defined in Clause 46. The XGMII has an optional physical instantiation. An optional physical instantiation of the PMA service interface has also been defined (see Clause 51). It is called XSBI (10 Gigabit Sixteen Bit Interface). Figure 49–4 depicts the relationship and mapping of the services provided by all of the interfaces relevant to 10GBASE-R.

The upper interface of the PCS may connect to the Reconciliation Sublayer through the XGMII or the PCS may connect to an XGXS sublayer. The XGXS and the Reconciliation Sublayer provide the same service interface to the PCS. The lower interface of the PCS may connect to the WIS to support a WAN PMD or to the PMA sublayer to support a 10GBASE-R LAN PMD. The WIS and PMA interfaces are functionally

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

equivalent except for data rate. When the PCS is connected directly to a LAN PMA, the nominal rate of the PMA service interface is 644.53125 Mtransfers/s, which provides capacity for the MAC data rate of 10 Gb/s. When the PCS is connected to a WAN PMA, the nominal rate of the WIS service interface is 599.04 Mtransfers/s and the MAC uses IFS stretch mode to ensure that there will be enough idle time that the PCS can delete idles to adjust to the lower rate. Since the data rates are different, WIS and PMA interface connections pose somewhat different constraints. The PCS shall support connection to either a WIS or to a PMA and may optionally support both.

If the optional Energy-Efficient Ethernet (EEE) capability is supported (see Clause 78, 78.3) then the interface with the PMA sublayer (or FEC sublayer) includes rx_mode and tx_mode to control power states in lower sublayers and energy_detect that indicates whether the PMD sublayer has detected a signal at the receiver. If the PHY includes an FEC sublayer, the interface includes rx_lpi_active to indicate that the LPI receive state diagram is not in RX_ACTIVE state.

It is important to note that, while this specification defines interfaces in terms of bits, octets, and frames, implementations may choose other data-path widths for implementation convenience.

49.1.6 Functional block diagram

Figure 49–4 provides a functional block diagram of the 10GBASE-R PHY.

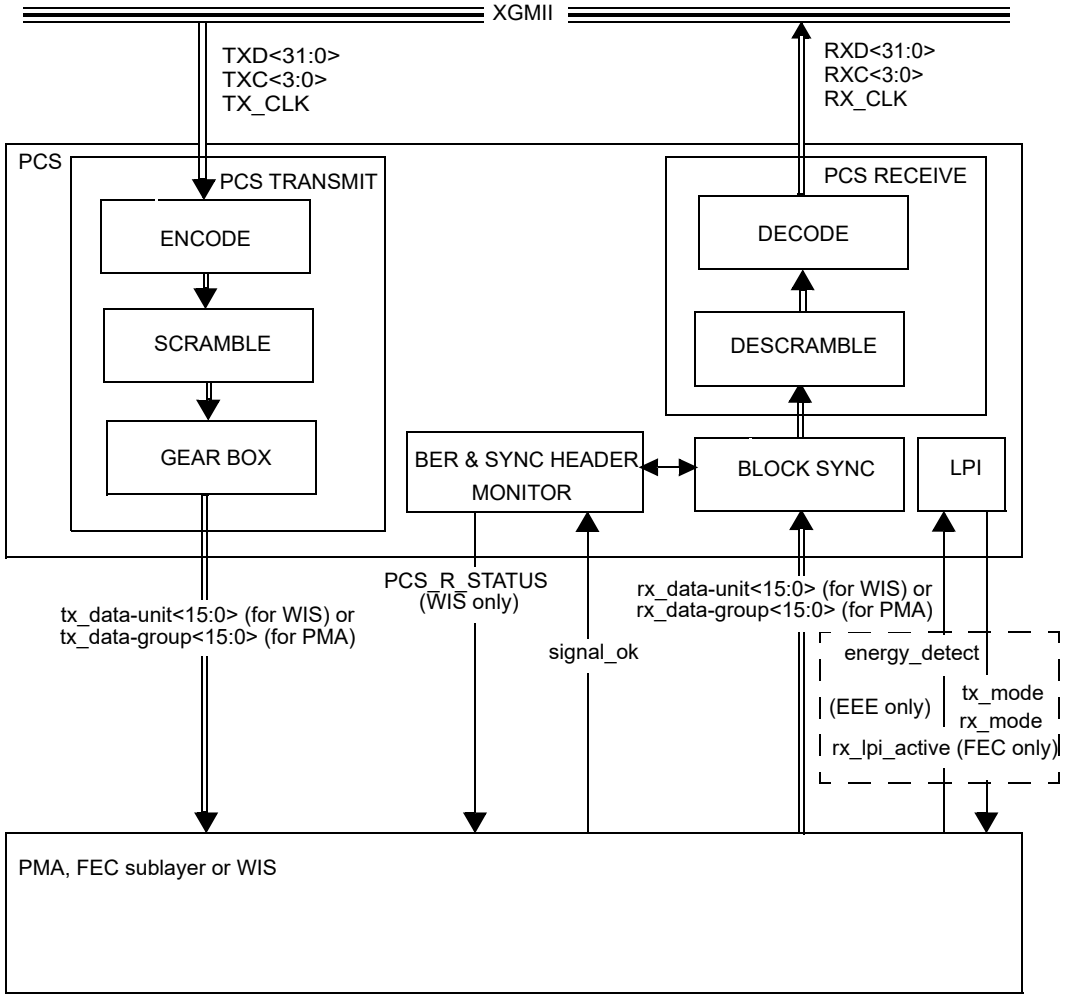


Figure 49–4—Functional block diagram

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.2 Physical Coding Sublayer (PCS)

49.2.1 PCS service interface (XGMII)

The PCS service interface allows the 10GBASE-R PCS to transfer information to and from a PCS client. A PCS client is generally the Reconciliation Sublayer or an XGXS sublayer. The PCS Interface is precisely defined as the 10 Gigabit Media Independent Interface (XGMII) in Clause 46.

49.2.2 Functions within the PCS

The PCS comprises the PCS Transmit, Block Synchronization, PCS Receive, and BER monitor processes for 10GBASE-R. The PCS shields the Reconciliation Sublayer (and MAC) from the specific nature of the underlying channel. The PCS transmit channel and receive channel can each operate in normal mode or, when not attached to a WIS, test-pattern mode. When the PCS is attached to a WIS, the WIS provides the test-pattern functionality.

When communicating with the XGMII, the PCS uses a four octet-wide, synchronous data path, with packet delimiting being provided by transmit control signals (TXCn = 1) and receive control signals (RXCn = 1). When communicating with the PMA, WIS, or FEC, the PCS uses a 16-bit wide, synchronous data path that conveys 16 encoded bits. Alignment to 64B/66B block is performed in the PCS. The WIS, PMA, and FEC sublayers operate independent of block and packet boundaries. The PCS provides the functions necessary to map packets between the XGMII format and the PMA service interface format.

When the transmit channel is in normal mode, the PCS Transmit process continuously generates blocks based upon the TXD <31:0> and TXC <3:0> signals on the XGMII. The Gearbox function of the PCS Transmit process then packs the resulting bits into 16-bit transmit data-units. Transmit data-units are sent to the PMA, WIS, or FEC service interface via the PMA_UNITDATA.request, WIS_UNITDATA.request, or FEC_UNITDATA.request primitive, respectively. When the WIS is present, the PCS Transmit process also adapts between the XGMII and WIS data rates by deleting idle characters.

When the transmit channel is in test-pattern mode, a test pattern is packed into the transmit data-units that are sent to the PMA service interface via the PMA_UNITDATA.request primitive.

When the receive channel is in normal mode, the PCS Synchronization process continuously monitors PMA_SIGNAL.indication(SIGNAL_OK), WIS_SIGNAL.indication(SIGNAL_OK), or FEC_SIGNAL.indication(SIGNAL_OK). When SIGNAL_OK indicates OK, then the PCS Synchronization process accepts data-units via the PMA_UNITDATA.indication primitive, the WIS_UNITDATA.indication primitive, or the FEC_UNITDATA.indication primitive. It attains block synchronization based on the 2-bit synchronization headers and conveys received blocks to the PCS Receive process. The PCS Synchronization process sets the sync_status flag to indicate whether the PCS has obtained synchronization.

When the PCS Synchronization process has obtained synchronization, the BER monitor process monitors the signal quality asserting hi_ber if excessive errors are detected. When sync_status is asserted and hi_ber is de-asserted, the PCS Receive process continuously accepts blocks. The PCS Receive process monitors these blocks and generates RXD <31:0> and RXC <3:0> on the XGMII. When the WIS is present, the PCS Receive process also adapts between the WIS and XGMII data rates by inserting idle characters. The PCS Receive process also sends the WIS_SIGNAL.request(PCS_R_STATUS) primitive to the WIS to indicate its status. The primitive is sent when the value of PCS_R_STATUS changes. The value of PCS_R_STATUS shall be FAIL when the Receive state diagram is in the RX_INIT state and OK otherwise.

When the receive channel is in test-pattern mode, the BER monitor process is disabled. The Receive process will be held in the RX_INIT state. The received bits will be compared to the test pattern and errors counted.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

A PCS that supports direct connection to a PMA shall provide transmit test-pattern mode for the square wave and pseudo-random patterns, and shall provide receive test-pattern mode for the pseudo-random pattern. The PCS may provide support for the PRBS9 transmit test pattern. It may provide support for the PRBS31 test pattern. Support of the optional PRBS31 test pattern shall include both the transmit and the receive capability for that pattern. Test-pattern mode is activated separately for transmit and receive. A PCS that supports direct connection to a PMA shall support transmit test-pattern mode and receive test-pattern mode operating simultaneously (if applicable) so as to support loopback testing. Test-pattern mode is provided by the WIS when a WIS is present.

49.2.3 Use of blocks

The PCS maps XGMII signals into 66-bit blocks, and vice versa, using a 64B/66B coding scheme. The synchronization headers of the blocks allow establishment of block boundaries by the PCS Synchronization process. Blocks are unobservable and have no meaning outside the PCS. The PCS functions ENCODE and DECODE generate, manipulate, and interpret blocks as provided by the rules in 49.2.4.

49.2.4 64B/66B transmission code

The PCS uses a transmission code to improve the transmission characteristics of information to be transferred across the link and to support transmission of control and data characters. The encodings defined by the transmission code ensure that sufficient transitions are present in the PHY bit stream to make clock recovery possible at the receiver. The encoding also preserves the likelihood of detecting any single or multiple bit errors that may occur during transmission and reception of information. In addition, the synchronization headers of the code enable the receiver to achieve block alignment on the incoming PHY bit stream. The 64B/66B transmission code specified for use in this standard has a high transition density and is a run-length-limited code.

The relationship of block bit positions to XGMII, PMA, and other PCS constructs is illustrated in Figure 49–5 for transmit and Figure 49–6 for receive. These figures illustrate the processing of a block containing 8 data octets. See 49.2.4.3 for information on how blocks containing control characters are mapped. Note that the sync header is generated by the encoder and bypasses the scrambler.

49.2.4.1 Notation conventions

For values shown as binary, the leftmost bit is the first transmitted bit.

64B/66B encodes 8 data octets or control characters into a block. Blocks containing control characters also contain a block type field. Data octets are labeled D_0 to D_7. Control characters other than /O/, /S/ and /T/ are labeled C_0 to C_7. The control character for ordered set is labeled as O_0 or O_4 since it is only valid on the first octet of the XGMII. The control character for start is labeled as S_0 or S_4 for the same reason. The control character for terminate is labeled as T_0 to T_7.

Two consecutive XGMII transfers provide eight characters that are encoded into one 66-bit transmission block. The subscript in the above labels indicates the position of the character in the eight characters from the XGMII transfers.

Contents of block type fields, data octets and control characters are shown as hexadecimal values. The LSB of the hexadecimal value represents the first transmitted bit. For instance, the block type field 0x1E is sent from left to right as 01111000. The bits of a transmitted or received block are labeled TxB<65:0> and RxB<65:0> respectively where TxB<0> and RxB<0> represent the first transmitted bit. The value of the sync header is shown as a binary value. Binary values are shown with the first transmitted bit (the LSB) on the left.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

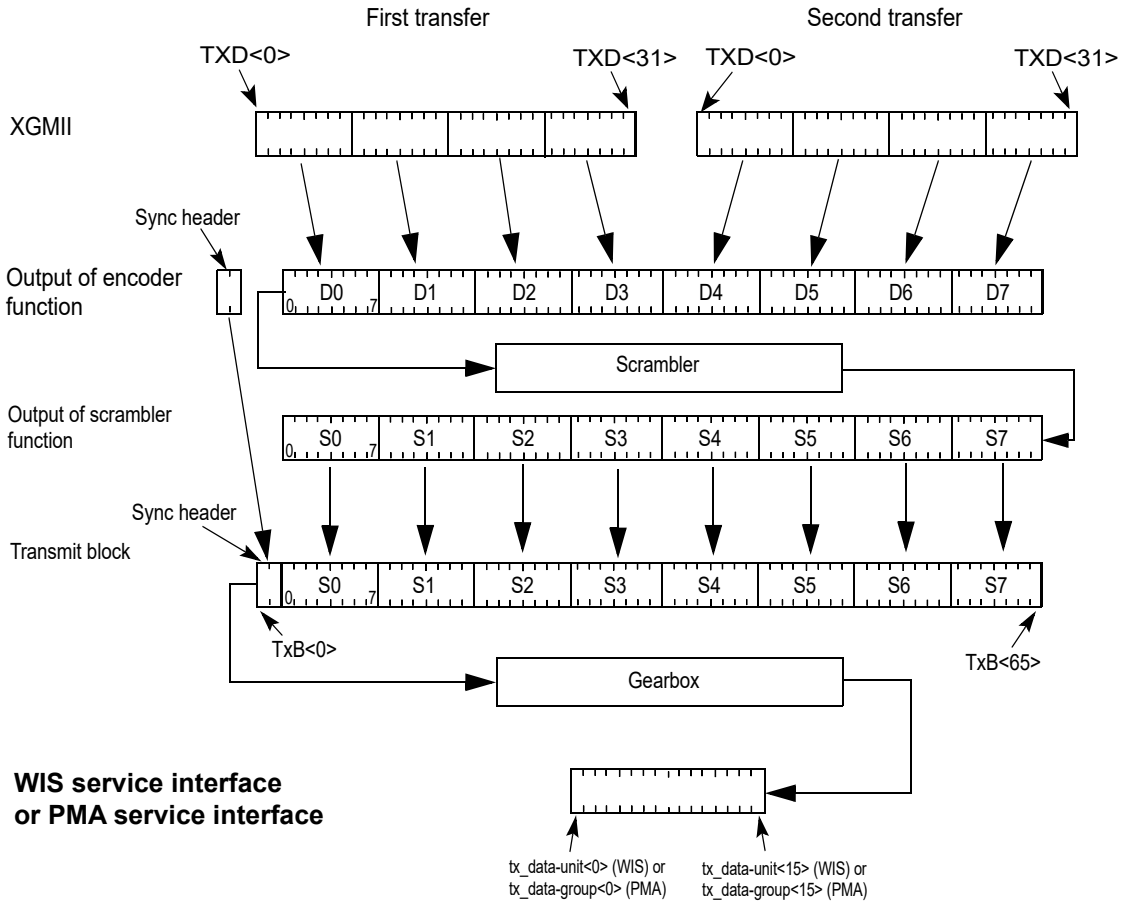


Figure 49–5—PCS Transmit bit ordering

49.2.4.2 Transmission order

Block bit transmission order is illustrated in Figure 49–5 and Figure 49–6. Note that these figures show the mapping from XGMII to 64B/66B block for a block containing eight data characters.

49.2.4.3 Block structure

Blocks consist of 66 bits. The first two bits of a block are the synchronization header (sync header). Blocks are either data blocks or control blocks. The sync header is 01 for data blocks and 10 for control blocks. Thus, there is always a transition between the first two bits of a block. The remainder of the block contains the payload. The payload is scrambled and the sync header bypasses the scrambler. Therefore, the sync header is the only position in the block that always contains a transition. This feature of the code is used to obtain block synchronization.

Data blocks contain eight data characters. Control blocks begin with an 8-bit block type field that indicates the format of the remainder of the block. For control blocks containing a Start or Terminate character, that character is implied by the block type field. Other control characters are encoded in a 7-bit control code or a 4-bit O Code. Each control block contains eight characters.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

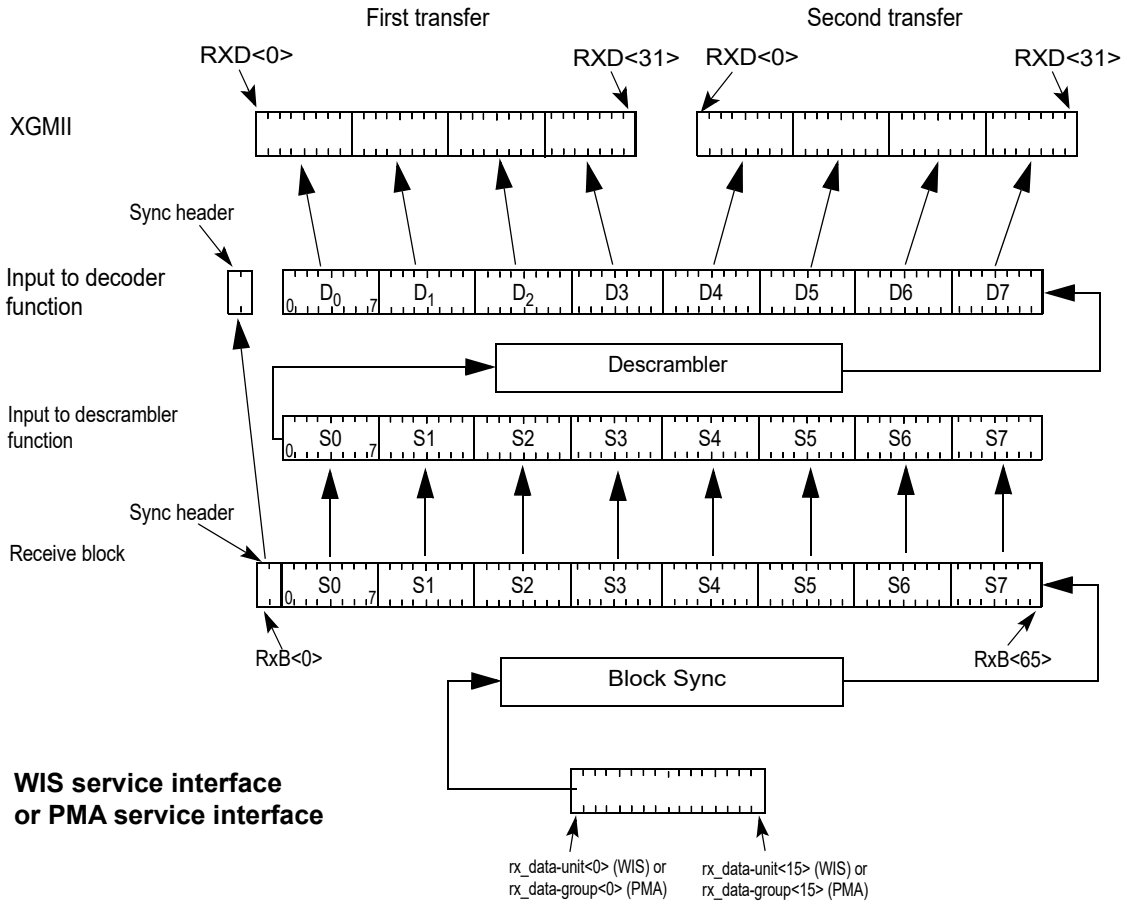


Figure 49–6—PCS Receive bit ordering

The format of the blocks is as shown in Figure 49–7. In the figure, the column labeled Input Data shows, in abbreviated form, the eight characters used to create the 66-bit block. These characters are either data characters or control characters and, when transferred across the XGMII interface, the corresponding TXC or RXC bit is set accordingly. Within the Input Data column, D_0 through D_7 are data octets and are transferred with the corresponding TXC or RXC bit set to zero. All other characters are control octets and are transferred with the corresponding TXC or RXC bit set to one. The single bit fields (thin rectangles with no label in the figure) are sent as zero and ignored upon receipt.

Bits and field positions are shown with the least significant bit on the left. Hexadecimal numbers are shown in normal hexadecimal. For example the block type field 0x1E is sent as 01111000 representing bits 2 through 9 of the 66 bit block. The least significant bit for each field is placed in the lowest numbered position of the field.

All unused values of block type field[93] are reserved.

[93]The block type field values have been chosen to have a 4-bit Hamming distance between them. The only unused value that maintains the Hamming distance is 0x00.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.2.4.4 Control codes

The same set of control characters are supported by the XGMII and the 10GBASE-R PCS. The representations of the control characters are the control codes. XGMII encodes a control character into an octet (an eight bit value). The 10GBASE-R PCS encodes the start and terminate control characters implicitly by the block type field. The 10GBASE-R PCS encodes the ordered set control codes using a combination of the block type field and a 4-bit O code for each ordered set. The 10GBASE-R PCS encodes each of the other control characters into a 7-bit C code.

The control characters and their mappings to 10GBASE-R control codes and XGMII control codes are specified in Table 49–1. All XGMII and 10GBASE-R control code values that do not appear in the table shall not be transmitted and shall be treated as an error if received. The ability to transmit or receive Low Power Idle (LPI) is required for PHYs that support EEE (see Clause 78). If EEE is not supported, LPI shall not be transmitted and shall be treated as an error if received.

Input Data	**Sync**	**Block Payload**							
Bit Position: / **Data Block Format:**	0 1	2							65
$D_0\ D_1\ D_2\ D_3/D_4\ D_5\ D_6\ D_7$	01	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
Control Block Formats:		**Block Type Field**							
$C_0\ C_1\ C_2\ C_3/C_4\ C_5\ C_6\ C_7$	10	0x1E	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7
$C_0\ C_1\ C_2\ C_3/O_4\ D_5\ D_6\ D_7$	10	0x2D	C_0	C_1	C_2	C_3	O_4	D_5	D_6	D_7
$C_0\ C_1\ C_2\ C_3/S_4\ D_5\ D_6\ D_7$	10	0x33	C_0	C_1	C_2	C_3		D_5	D_6	D_7
$O_0\ D_1\ D_2\ D_3/S_4\ D_5\ D_6\ D_7$	10	0x66	D_1	D_2	D_3	O_0		D_5	D_6	D_7
$O_0\ D_1\ D_2\ D_3/O_4\ D_5\ D_6\ D_7$	10	0x55	D_1	D_2	D_3	O_0	O_4	D_5	D_6	D_7
$S_0\ D_1\ D_2\ D_3/D_4\ D_5\ D_6\ D_7$	10	0x78	D_1	D_2	D_3	D_4		D_5	D_6	D_7
$O_0\ D_1\ D_2\ D_3/C_4\ C_5\ C_6\ C_7$	10	0x4B	D_1	D_2	D_3	O_0	C_4	C_5	C_6	C_7
$T_0\ C_1\ C_2\ C_3/C_4\ C_5\ C_6\ C_7$	10	0x87		C_1	C_2	C_3	C_4	C_5	C_6	C_7
$D_0\ T_1\ C_2\ C_3/C_4\ C_5\ C_6\ C_7$	10	0x99	D_0		C_2	C_3	C_4	C_5	C_6	C_7
$D_0\ D_1\ T_2\ C_3/C_4\ C_5\ C_6\ C_7$	10	0xAA	D_0	D_1		C_3	C_4	C_5	C_6	C_7
$D_0\ D_1\ D_2\ T_3/C_4\ C_5\ C_6\ C_7$	10	0xB4	D_0	D_1	D_2		C_4	C_5	C_6	C_7
$D_0\ D_1\ D_2\ D_3/T_4\ C_5\ C_6\ C_7$	10	0xCC	D_0	D_1	D_2	D_3		C_5	C_6	C_7
$D_0\ D_1\ D_2\ D_3/D_4\ T_5\ C_6\ C_7$	10	0xD2	D_0	D_1	D_2	D_3	D_4		C_6	C_7
$D_0\ D_1\ D_2\ D_3/D_4\ D_5\ T_6\ C_7$	10	0xE1	D_0	D_1	D_2	D_3	D_4	D_5		C_7
$D_0\ D_1\ D_2\ D_3/D_4\ D_5\ D_6\ T_7$	10	0xFF	D_0	D_1	D_2	D_3	D_4	D_5	D_6	

Figure 49–7—64B/66B block formats

49.2.4.5 Ordered sets

Ordered sets are used to extend the ability to send control and status information over the link such as remote fault and local fault status. Ordered sets consist of a control character followed by three data characters. Ordered sets always begin on the first octet of the XGMII. 10 Gigabit Ethernet uses one kind of ordered set: the sequence ordered set (see 46.3.4). The sequence ordered set control character is denoted /Q/. An

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

additional ordered set, the signal ordered set, has been reserved and it begins with another control code. The 4-bit O field encodes the control code. See Table 49–1 for the mappings.

49.2.4.6 Valid and invalid blocks

A block is invalid if any of the following conditions exists:

a) The sync field has a value of 00 or 11.
b) The block type field contains a reserved value.
c) Any control character contains a value not in Table 49–1.
d) Any O code contains a value not in Table 49–1, or the O code 0xF is received on an Ethernet PHY.
e) The set of eight XGMII characters does not have a corresponding block format in Figure 49–7.

49.2.4.7 Idle (/I/) and Low Power Idle (/LI/)

Idle control characters (/I/) are transmitted when idle control characters are received from the XGMII. Idle characters may be added or deleted by the PCS to adapt between clock rates. /I/ insertion and deletion shall occur in groups of 4. /I/s may be added following idle or ordered sets. They shall not be added while data is being received. When deleting /I/s, the first four characters after a /T/ shall not be deleted.

To communicate LPI, LPI control character /LI/ is sent continuously in place of /I/. LPI control characters are transmitted when LPI control characters are received from the XGMII. LPI characters may be added or deleted by the PCS to adapt between clock rates in a similar manner to idle control characters. /LI/ insertion and deletion shall occur in groups of four. /LI/s may only be added following other LPI characters.

Table 49–1—Control codes

Control character	Notation	XGMII Control Code	10GBASE-R Control Code	10GBASE-R O Code	8B/10B Code[a]
idle	/I/	0x07	0x00		K28.0 or K28.3 or K28.5
LPI	/LI/	0x06	0x06		K28.0 with D20.5 in one row or K28.3 or K28.5 with D20.5 in one row[b]
start	/S/	0xFB	Encoded by block type field		K27.7
terminate	/T/	0xFD	Encoded by block type field		K29.7
error	/E/	0xFE	0x1E		K30.7
Sequence ordered set	/Q/	0x9C	Encoded by block type field plus O code	0x0	K28.4
reserved0	/R/[c]	0x1C	0x2D		K28.0
reserved1		0x3C	0x33		K28.1
reserved2	/A/	0x7C	0x4B		K28.3
reserved3	/K/	0xBC	0x55		K28.5

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

Table 49–1—Control codes *(continued)*

Control character	Notation	XGMII Control Code	10GBASE-R Control Code	10GBASE-R O Code	8B/10B Code[a]
reserved4		0xDC	0x66		K28.6
reserved5		0xF7	0x78		K23.7
Signal ordered set[d]	/Fsig/	0x5C	Encoded by block type field plus O code	0xF	K28.2

[a]For information only. The 8B/10B code is specified in Clause 36. Usage of the 8B/10B code for 10 Gb/s operation is specified in Clause 48.
[b]See 48.2.4.2.
[c]The codes for /A/, /K/, and /R/ are used on the XAUI interface to signal idle. They are not present on the XGMII when no errors have occurred, but certain bit errors cause the XGXS to send them on the XGMII.
[d]Used by INCITS T11 Fibre Channel.

49.2.4.8 Start (/S/)

The start control character (/S/) indicates the start of a packet. This delimiter is only valid on the first octet of the XGMII (TXD<0:7> and RXD<0:7>). Receipt of an /S/ on any other octet of TxD indicates an error. Block type field values implicitly encode an /S/ as the fifth or first character of the block. These are the only characters of a block on which a start can occur.

49.2.4.9 Terminate (/T/)

The terminate control character (/T/) indicates the end of a packet. Since packets may be any length, the /T/ can occur on any octet of the XGMII interface and on any character of the block. The location of the /T/ in the block is implicitly encoded in the block type field. A valid end of packet occurs when a block containing a /T/ is followed by a control block that does not contain a /T/.

49.2.4.10 ordered set (/O/)

The ordered set control characters (/O/) indicate the start of an ordered set. There are two kinds of ordered sets: the sequence ordered set and the signal ordered set (which is reserved). When it is necessary to designate the control character for the sequence ordered set specifically, /Q/ will be used. /O/ is only valid on the first octet of the XGMII. Receipt of an /O/ on any other octet of TXD indicates an error. Block type field values implicitly encode an /O/ as the first or fifth character of the block. The 4-bit O code encodes the specific /O/ character for the ordered set.

Sequence ordered sets may be deleted by the PCS to adapt between clock rates. Such deletion shall only occur when two consecutive sequence ordered sets have been received and shall delete only one of the two. Only Idles may be inserted for clock compensation. Signal ordered sets are not deleted for clock compensation.

49.2.4.11 Error (/E/)

The /E/ is sent whenever an /E/ is received. It is also sent when invalid blocks are received. The /E/ allows physical sublayers such as the XGXS and PCS to propagate received errors. See R_BLOCK_TYPE and T_BLOCK_TYPE function definitions in 49.2.13.2.3 for further information.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.2.5 Transmit process

The transmit process generates blocks based upon the TXD<31:0> and TXC<3:0> signals received from the XGMII. Two XGMII data transfers are encoded into each block. It takes 4.125 PMA_UNITDATA or WIS_UNITDATA transfers to send a block of data. Therefore, if the PCS is connected to an XGMII and PMA sublayer where the ratio of their transfer rates is exactly 16:33, then the transmit process does not need to perform rate adaptation. Where the XGMII and PMA sublayer data rates are not synchronized to that ratio, the transmit process will need to insert idles, delete idles, or delete sequence ordered sets to adapt between the rates. The WIS data rate is always slower than the XGMII data rate and a PCS connected to a WIS deletes idles or sequence ordered sets to adapt between rates.

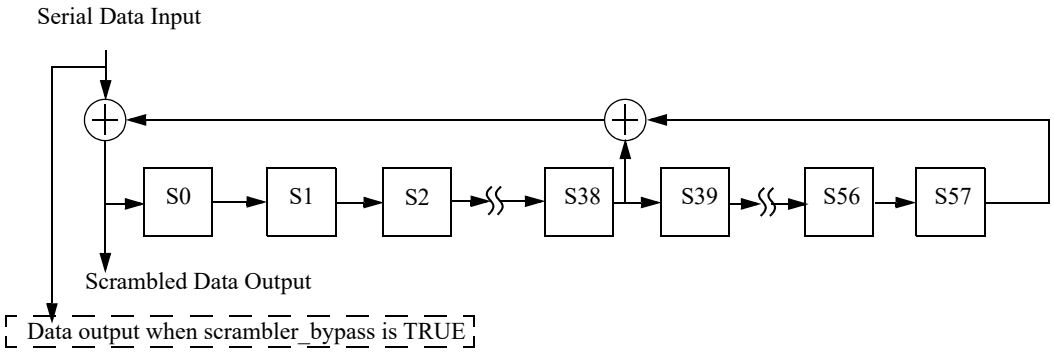
The transmit process generates blocks as specified in the transmit process state diagram. The contents of each block are contained in a vector tx_coded<65:0>, which is passed to the scrambler. tx_coded<1:0> contains the sync header and the remainder of the bits contain the block payload.

49.2.6 Scrambler

The payload of the block is scrambled with a self-synchronizing scrambler. The scrambler shall produce the same result as the implementation shown in Figure 49–8. This implements the scrambler polynomial:[94]

$$G(x) = 1 + x^{39} + x^{58} \tag{49–1}$$

There is no requirement on the initial value for the scrambler. The scrambler is run continuously on all payload bits. The sync header bits bypass the scrambler.



NOTE—Scrambler_bypass is only required to support EEE capability.

Figure 49–8—Scrambler

To aid block synchronization in the receiver for EEE capability when Clause 74 FEC is in use, when scrambler_bypass is TRUE the PCS shall pass the unscrambled data from the scrambler input rather than the scrambled data from the scrambler output. The scrambler shall continue to advance normally.

[94]The convention here, which considers the most recent bit into the scrambler to be the lowest order term, is consistent with most references and with other scramblers shown in this standard. Some references consider the most recent bit into the scrambler to be the highest order term and would therefore identify this as the inverse of the polynomial in Equation (49–1). In case of doubt, note that the conformance requirement is based on the representation of the scrambler in the figure rather than the polynomial equation.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.2.7 Gearbox

The gearbox adapts between the 66-bit width of the blocks and the 16-bit width of the PMA or WIS interface. It receives the 66-bit blocks. When the transmit channel is operating in normal mode, the gearbox sends 16 bits of transmit data at a time via WIS_UNITDATA.request or PMA_UNITDATA.request primitives. The UNITDATA.request primitives are fully packed with bits. For example, if one block happened to start with the sync header on bits 0 and 1 of a PMA_UNITDATA.request, then the last two bits of that block would be on bits 0 and 1 of a PMA_UNITDATA.request and the next block would begin with a sync header on bits 2 and 3 of that PMA_UNITDATA.request. When a PMA_UNITDATA.request or WIS_UNITDATA.request contains bits from two blocks, then the bits from the first block shall be placed in the lowest numbered bits of tx_data-group<15:0>. The bits shall be packed into the tx_data-group in sequence with the lowest numbered bit of the block going into the lowest numbered bit of the part of tx_data-group<15:0> bits containing bits from that block (see Figure 49–5).

The gearbox functionality is necessary when the optional PMA compatibility interface, XSBI, is implemented since that interface passes data over a 16-bit wide path. It is also necessary when connecting to a WIS since the WIS processes the data stream with 8-bit granularity. When neither the WIS nor the XSBI is implemented, the internal data-path width between the PCS and PMA is an implementation choice. Depending on the path width, the gearbox functionality may not be necessary.

49.2.8 Test-pattern generators

When the transmit channel is operating in test-pattern mode, it sends 16 bits of test pattern at a time via PMA_UNITDATA.request primitives. When the PCS allows direct connection to the PMA, the test-pattern generator shall be implemented. The test-pattern generator does not apply to a PCS, which only supports connection to the WIS. A PCS which supports both WIS and direct PMA attachment may reject or allow an attempt to activate a transmit test-pattern mode when a WIS is attached.

There are two types of required transmit test patterns: square wave and pseudo-random. The square wave pattern is intended to aid in conducting certain transmitter tests. It is not intended for receiver tests and the receiver is not expected to receive this test pattern. The pseudo-random test-pattern mode is suitable for receiver tests and for certain transmitter tests. There is an optional PRBS9 transmit test pattern that may be used for some transmitter tests. There is also an optional PRBS31 test pattern, which may be used for some transmit and receiver tests. When this option is supported, both the PRBS31 test-pattern generator and the PRBS31 test-pattern checker shall be provided. See 52.9 and 68.6 for recommendations on the appropriate pattern for tests.

When square wave pattern is selected, the PCS will send a repeating pattern of n ones followed by n zeros where n may be any number between 4 and 11 inclusive. The value of n is an implementation choice.

When pseudo-random pattern is selected, the test pattern is generated by the scrambler using the seeds loaded through the MDIO registers and the selected data pattern. The scrambler is loaded with a seed or its inverse at the start of a block every 128 blocks. The seeds are loaded in the following pattern:

Seed A
Seed A Invert
Seed B
Seed B Invert

Invert indicates that the seed is inverted for that load. Either 64 zeros or the 64-bit encoding for two Local Fault ordered sets can be selected as the data pattern. After loading Seed A or Seed B, the scrambler input shall be driven with the data pattern. After loading Seed A Invert or Seed B Invert, the scrambler input shall be driven with the inverse of the data pattern. While in pseudo-random test-pattern mode, the sync headers will be the control sync header, 10. Thus the pseudo-random test pattern is a series of blocks with the control

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

sync header and a pseudo-random payload. The characteristics of the pseudo-random test pattern can be varied by varying the seed values and data input.

When the optional PRBS31 mode is selected, the PRBS31 pattern generator sends 16 bits of PRBS31 test pattern at a time via PMA_UNITDATA.request primitives. The PRBS31 test pattern is the output of a Pseudo-Random Bit Sequence of order 31 (PRBS31) generator. The PRBS31 pattern generator shall produce the same result as the implementation shown in Figure 49–9. This implements the inverted version of the bit stream produced by the polynomial shown in Equation (49–2):[95]

$$G(x) = 1 + x^{28} + x^{31} \tag{49–2}$$

The initial value of the PRBS31 pattern generator shall not be all zeros. It may be any other value.

The optional PRBS9 pattern is defined in Table 68–6.

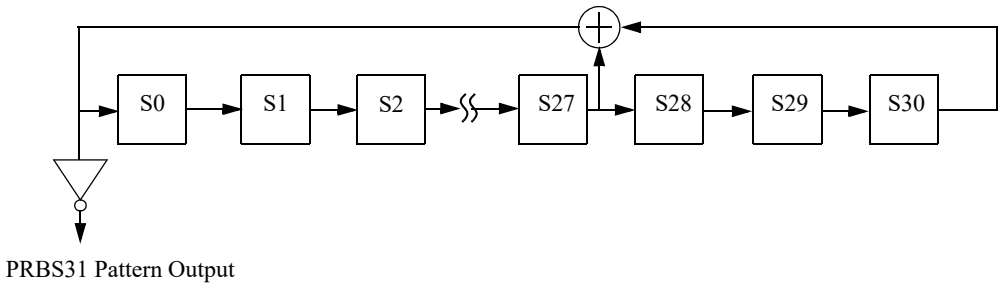


Figure 49–9—PRBS31 pattern generator

49.2.9 Block synchronization

When the receive channel is operating in normal mode, the block synchronization function receives data via 16-bit PMA_UNITDATA.request or WIS_UNITDATA.request primitives. It shall form a bit stream from the primitives by concatenating requests with the bits of each primitive in order from rx_data-group<0> to rx_data-group<15> (see Figure 49–6). It obtains lock to the 66-bit blocks in the bit stream using the sync headers and outputs 66-bit blocks. Lock is obtained as specified in the block lock state diagram shown in Figure 49–14.

If EEE is not supported then block_lock is identical to rx_block_lock. Otherwise the relationship between block_lock and rx_block_lock is given by Figure 49–13.

49.2.10 Descrambler

The descrambler processes the payload to reverse the effect of the scrambler using the same polynomial. It shall produce the same result as the implementation shown in Figure 49–10.

49.2.11 Receive process

The receive process decodes blocks to produce RXD<31:0> and RXC<3:0> for transmission to the XGMII. Two XGMII data transfers are decoded from each block. Where the XGMII and PMA sublayer data rates are not synchronized to a 16:33 ratio, the receive process will insert idles, delete idles, or delete sequence

[95]The convention here, which considers the most recent bit into the PRBS generator to be the lowest order term, is consistant with most references and with other scramblers shown in this standard. Some references consider the most recent bit into the PRBS generator to be the highest order term and would therefore identify this as the inverse of the polynomial in Equation (49–2). In case of doubt, note that the conformance requirement is based on the representation in the figure rather than the polynomial equation.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

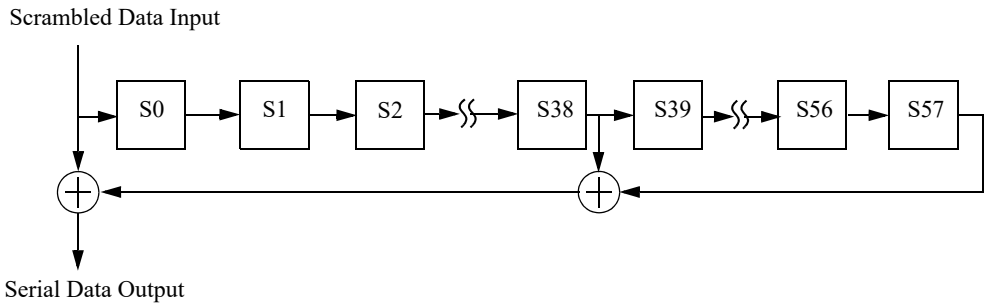


Figure 49–10—Descrambler

ordered sets to adapt between rates. The WIS data rate is always slower than the XGMII data rate and a PCS connected to a WIS will insert idles to adapt between the rates.

The receive process decodes blocks as specified in the receive state diagram shown in Figure 49–17.

49.2.12 Test-pattern checker

When the PCS allows direct connection to the PMA, the pseudo-random test-pattern checker shall be implemented. The pseudo-random test-pattern checker does not apply to a PCS, which only supports connection to the WIS. When the PRBS31 test-pattern option is supported, the PRBS31 receive test-pattern checker shall be implemented. A PCS that supports both WIS and direct PMA attachment may reject or allow an attempt to activate a receive test-pattern mode when a WIS is attached.

When the receive channel is operating in pseudo-random test-pattern mode, the pseudo-random test-pattern checker checks the bits received via PMA_UNITDATA.indication primitives.

The pseudo-random test-pattern checker utilizes the lock state diagram and the descrambler operating as they do during normal data reception. The hi_ber state diagram is disabled during receive test-pattern mode. When block_lock is true and the pseudo-random receive test-pattern mode is active, the pseudo-random test-pattern checker observes the output from the descrambler. When the output of the descrambler is the data pattern or its inverse, a match is detected. Since the transmitter's scrambler is loaded with a seed value every 128 blocks and the receiver's descrambler is running normally, a mismatch will be detected once every 128 blocks in the absence of errors. The pseudo-random test-pattern checker will count 128-block windows. When operating in pseudo-random test pattern, the test-pattern error counter counts blocks with a mismatch corrected to remove the effect of loading a new seed. The first block with a mismatch in a window shall not increment the test-pattern error counter. Any subsequent block with a mismatch in a window indicates an error and shall increment the test-pattern error counter.

When the receive channel is operating in PRBS31 test mode, the PRBS31 pattern checker checks the bits received in PMA_UNITDATA.indication primitives relative to the PRBS31 test pattern.

The PRBS31 pattern error checker is self-synchronizing. It compares each bit received to the result of the PRBS31 generator based on the prior 31 bits received. It shall produce the same result as the implementation shown in Figure 49–11. When no errors occur, the PRBS31 pattern error signal will be zero. When an isolated bit error occurs, it will cause the PRBS31 pattern error signal to go high three times; once when it is

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

received and once when it is at each tap. The test-pattern error counter shall increment once for each bit time that the PRBS31 pattern error signal is high.

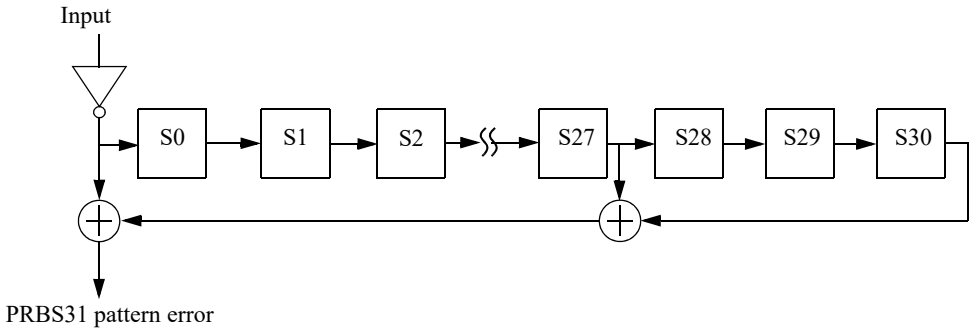


Figure 49–11—PRBS31 pattern checker

Note that the test-pattern error counter behavior is dependent on the test-pattern mode. In pseudo-random test mode, it is counting block errors. In PRBS31 test mode, it is counting bit errors at the PRBS31 pattern checker output.

49.2.13 Detailed functions and state diagrams

49.2.13.1 State diagram conventions

The body of this subclause is composed of state diagrams, including the associated definitions of variables, constants, and functions. Should there be a discrepancy between a state diagram and descriptive text, the state diagram prevails.

The notation used in the state diagrams follows the conventions of 21.5. State diagram timers follow the conventions of 14.2.3.2. The notation ++ after a counter or integer variable indicates that its value is to be incremented.

49.2.13.2 State variables

49.2.13.2.1 Constants

EBLOCK_R<71:0>
: 72 bit vector to be sent to the XGMII interface containing /E/ in all the eight character locations.

EBLOCK_T<65:0>
: 66 bit vector to be sent to the PMA containing /E/ in all the eight character locations.

LBLOCK_R<71:0>
: 72 bit vector to be sent to the XGMII interface containing two Local Fault ordered sets. The Local Fault ordered set is defined in 46.3.4.

LBLOCK_T<65:0>
: 66 bit vector to be sent to the PMA containing two Local Fault ordered sets.

49.2.13.2.2 Variables

ber_test_sh
: Boolean variable that is set true when a new sync header is available for testing and false when BER_TEST_SH state is entered. A new sync header is available for testing when the Block Sync process has accumulated enough bits from the PMA or the WIS to evaluate the header of the next block

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

block_lock
: Boolean variable that is set true when receiver acquires block delineation

NOTE—If the EEE capability is supported, then this variable is affected by the LPI receive state diagram. If the EEE capability is not supported then this variable is identical to rx_block_lock controlled by the lock state diagram.

hi_ber
: Boolean variable which is asserted true when the ber_cnt equals 16 indicating a bit error ratio $>10^{-4}$

reset
: Boolean variable that controls the resetting of the PCS. It is true whenever a reset is necessary including when reset is initiated from the MDIO, during power on, and when the MDIO has put the PCS into low-power mode.

r_test_mode
: Boolean variable that is asserted true when the receiver is in test-pattern mode.

rx_block_lock
: Variable used by the lock state diagram to reflect the status of the code-group delineation. This variable is set TRUE when the receiver acquires block delineation.

rx_coded<65:0>
: Vector containing the input to the 64B/66B decoder. The format for this vector is shown in Figure 49–7. The leftmost bit in the figure is rx_coded<0> and the rightmost bit is rx_coded<65>.

rx_raw<71:0>
: Vector containing two successive XGMII transfers. RXC<0> through RXC<3> for the first transfer are placed in rx_raw<0> through rx_raw<3>, respectively. RXC<0> through RXC<3> for the second transfer are placed in rx_raw<4> through rx_raw<7>, respectively. RXD<0> through RXD<31> for the first transfer are placed in rx_raw<8> through rx_raw<39>, respectively. RXD<0> through RXD<31> for the second transfer are placed in rx_raw<40> through rx_raw<71>, respectively.

sh_valid
: Boolean indication that is set true if received block rx_coded has valid sync header bits. That is, sh_valid is asserted if rx_coded<0> ≠ rx_coded<1> and de-asserted otherwise.

signal_ok
: Boolean variable that is set based on the most recently received value of PMA_SIGNAL.indication(SIGNAL_OK), WIS_SIGNAL.indication(SIGNAL_OK), or FEC_SIGNAL.indication(SIGNAL_OK). It is true if the value was OK and false if the value was FAIL.

slip_done
: Boolean variable that is asserted true when the SLIP requested by the Block Lock state diagram has been completed indicating that the next candidate block sync position can be tested.

test_sh
: Boolean variable that is set true when a new sync header is available for testing and false when TEST_SH state is entered. A new sync header is available for testing when the Block Sync process has accumulated enough bits from the PMA or the WIS to evaluate the header of the next block

tx_coded<65:0>
: Vector containing the output from the 64B/66B encoder. The format for this vector is shown in Figure 49–7. The leftmost bit in the figure is tx_coded<0> and the rightmost bit is tx_coded<65>.

tx_raw<71:0>
: Vector containing two successive XGMII transfers. TXC<0> through TXC<3> for the first transfer are placed in tx_raw<0> through tx_raw<3>, respectively. TXC<0> through TXC<3> for the second transfer are placed in tx_raw<4> through tx_raw<7>, respectively. TXD<0> through TXD<31> for the first transfer are placed in tx_raw<8> through tx_raw<39>, respectively. TXD<0> through TXD<31> for the second transfer are placed in tx_raw<40> through tx_raw<71>, respectively.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

The following variables are used only for the EEE capability:

energy_detect
: A Boolean variable sent from the PMD that is set to TRUE when signal energy is detected at the receiver and is set to FALSE otherwise

rx_lpi_active
: A Boolean variable that is set to TRUE when the receiver is in a low power state and set to FALSE when it is in an active state and capable of receiving data.

rx_mode
: A variable set to QUIET while the receiver is in the RX_QUIET state and set to DATA otherwise

tx_mode
: A variable set to QUIET when the transmitter is in the TX_QUIET state, set to ALERT when the transmitter is in the TX_ALERT state, and set to DATA otherwise. When set to QUIET, the PMD disables the transmitter as described in 72.6.5. When set to ALERT, the PMD transmits a repeating pattern of eight ones and eight zeroes as described in 72.6.2. When set to DATA the PMD passes data as normal.

scrambler_bypass
: This Boolean variable is used to bypass the Tx PCS scrambler in order to assist rapid synchronization following low power idle. When set to TRUE, the PCS will pass the unscrambled data from the scrambler input rather than the scrambled data from the scrambler output. The scrambler will continue to operate normally, shifting input data into the delay line. When scrambler_bypass is set to FALSE the PCS will pass scrambled data from the scrambler output.

scr_bypass_enable
: A Boolean variable used to indicate to the transmit LPI state diagram that the scrambler bypass option is required. The PHY shall set scr_bypass_enable = TRUE if Clause 74 FEC is in use. The PHY shall set scr_bypass_enable = FALSE if this FEC is not in use.

49.2.13.2.3 Functions

DECODE(rx_coded<65:0>)
: Decodes the 66-bit vector returning rx_raw<71:0> that is sent to the XGMII. The DECODE function shall decode the block as specified in 49.2.4.

ENCODE(tx_raw<71:0>)
: Encodes the 72-bit vector returning tx_coded<65:0> of which tx_coded<63:0> is sent to the scrambler. The two high order sync bits bypass the scrambler. The ENCODE function shall encode the block as specified in 49.2.4.

R_BLOCK_TYPE = {C, S, T, D, E, LI}
: This function classifies each 66-bit rx_coded vector as belonging to one of the following types depending on its contents.

  Values: C; The vector contains a sync header of 10 and one of the following:
  a) A block type field of 0x1E and eight valid control characters other than /E/; and, if the EEE capability is supported, zero or four of the characters are /LI/;
  b) A block type field of 0x2D or 0x4B, a valid O code, and four valid control characters;
  c) A block type field of 0x55 and two valid O codes.

  LI; For EEE capability, the LI type is supported where the vector contains a sync header of 10, a block type field of 0x1E, and eight control characters of 0x06 (/LI/).

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

S; The vector contains a sync header of 10 and one of the following:
a) A block type field of 0x33 and four valid control characters;
b) A block type field of 0x66 and a valid O code;
c) A block type field of 0x78.
T; The vector contains a sync header of 10, a block type field of 0x87, 0x99, 0xAA, 0xB4, 0xCC, 0xD2, 0xE1 or 0xFF and all control characters are valid.
D; The vector contains a sync header of 01.
E; The vector does not meet the criteria for any other value.

A valid control character is one containing a 10GBASE-R control code specified in Table 49–1. A valid O code is one containing an O code specified in Table 49–1.

NOTE—A PCS that does not support EEE classifies vectors containing one or more /LI/ control characters as type E.

R_TYPE(rx_coded<65:0>)

Returns the R_BLOCK_TYPE of the rx_coded<65:0> bit vector.

R_TYPE_NEXT

Prescient end of packet check function. It returns the R_BLOCK_TYPE of the rx_coded vector immediately following the current rx_coded vector.

SLIP

Causes the next candidate block sync position to be tested. The precise method for determining the next candidate block sync position is not specified and is implementation dependent. However, an implementation shall ensure that all possible bit positions are evaluated.

T_BLOCK_TYPE = {C, S, T, D, E, LI}

This function classifies each 72-bit tx_raw vector as belonging to one of the following types depending on its contents.

Values: C; The vector contains one of the following:
a) eight valid control characters other than /O/, /S/, /T/ and /E/; and, if the EEE capability is supported, zero or four of the characters are /LI/;
b) one valid ordered set and four valid control characters other than /O/, /S/ and /T/;
c) two valid ordered sets.
LI; For EEE capability, this vector contains eight /LI/ characters.
S; The vector contains an /S/ in its first or fifth character, any characters before the S character are valid control characters other than /O/, /S/ and /T/ or form a valid ordered set, and all characters following the /S/ are data characters.
T; The vector contains a /T/ in one of its characters, all characters before the /T/ are data characters, and all characters following the /T/ are valid control characters other than /O/, /S/ and /T/.
D; The vector contains eight data characters.
E; The vector does not meet the criteria for any other value.

A tx_raw character is a control character if its associated TXC bit is asserted. A valid control character is one containing an XGMII control code specified in Table 49–1. A valid ordered set consists of a valid /O/ character in the first or fifth characters and data characters in the three characters following the /O/. A valid /O/ is any character with a value for O code in Table 49–1.

NOTE—A PCS that does not support EEE, classifies vectors containing one or more /LI/ control characters as type E.

T_TYPE(tx_raw<71:0>)

Returns the T_BLOCK_TYPE of the tx_raw<71:0> bit vector.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.2.13.2.4 Counters

ber_cnt
: Count up to a maximum of 16 of the number of invalid sync headers within the current 125 μs period.

sh_cnt
: Count of the number of sync headers checked within the current 64 block window.

sh_invalid_cnt
: Count of the number of invalid sync headers within the current 64 block window.

The following counter is used only for the EEE capability.

wake_error_counter
: A counter that is incremented each time that the LPI receive state diagram enters the RX_WTF state indicating that a wake time fault has been detected. The counter is reflected in register 3.22 (see 45.2.3.12).

49.2.13.2.5 Timers

State diagram timers follow the conventions of 14.2.3.2.

125us_timer
: Timer that is triggered every 125 μs +1%, –25%.

The following timers are used only for the EEE capability:

one_us_timer
: A timer used to count approximately 1 μs intervals. The timer terminal count is set to T_{1U}. When the timer reaches terminal count it will set the one_us_timer_done = TRUE.

rx_tq_timer
: This timer is started when the PCS receiver enters the RX_SLEEP state. The timer terminal count is set to T_{QR}. When the timer reaches terminal count it will set the rx_tq_timer_done = TRUE.

rx_tw_timer
: This timer is started when the PCS receiver enters the RX_WAKE state. The timer terminal count shall be set to a value no larger than the maximum value given for T_{WR} in Table 49–3. When the timer reaches terminal count it will set the rx_tw_timer_done = TRUE.

rx_wf_timer
: This timer is started when the PCS receiver enters the RX_WTF state, indicating that the receiver has encountered a wake time fault. The rx_wf_timer allows the receiver an additional period in which to synchronize or return to the QUIET state before a link failure is indicated. The timer terminal count is set to T_{WTF}. When the timer reaches terminal count it will set the rx_wf_timer_done = TRUE.

tx_ts_timer
: This timer is started when the PCS transmitter enters the TX_SLEEP state. The timer terminal count is set to T_{SL}. When the timer reaches terminal count it will set the tx_ts_timer_done = TRUE.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

tx_tq_timer
: This timer is started when the PCS transmitter enters the TX_QUIET state. The timer terminal count is set to T_{QL}. When the timer reaches terminal count it will set the tx_tq_timer_done = TRUE.

tx_tw_timer
: This timer is started when the PCS transmitter enters the TX_WAKE state. The timer terminal count is set to T_{WL}. When the timer reaches terminal count it will set the tx_tw_timer_done = TRUE.

49.2.13.3 State diagrams

The Lock state diagram shown in Figure 49–14 determines when the PCS has obtained lock to the received data stream. The BER Monitor state diagram shown in Figure 49–15 monitors the received signal for high bit error ratio.

The Transmit state diagram shown in Figure 49–16 controls the encoding of transmitted blocks. It makes exactly one transition for each transmit block processed. Though the Transmit state diagram sends Local Fault ordered sets when reset is asserted, the scrambler and gearbox may not be operational during reset. Thus, the Local Fault ordered sets may not appear on the WIS or PMA service interface.

The Receive state diagram shown in Figure 49–17 controls the decoding of received blocks. It makes exactly one transition for each receive block processed.

The PCS shall perform the functions of Lock, BER Monitor, Transmit, and Receive as specified in these state diagrams, including the optional EEE capability if implemented.

49.2.13.3.1 LPI state diagrams

A PCS that supports the EEE capability shall implement the LPI transmit and receive processes as shown in Figure 49–12 and Figure 49–13. The transmit LPI state diagram controls tx_mode, which disables the transmitter when it is set to QUIET. The receive LPI state diagram controls block_lock during LPI and signals the end of LPI to the receive state diagram.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

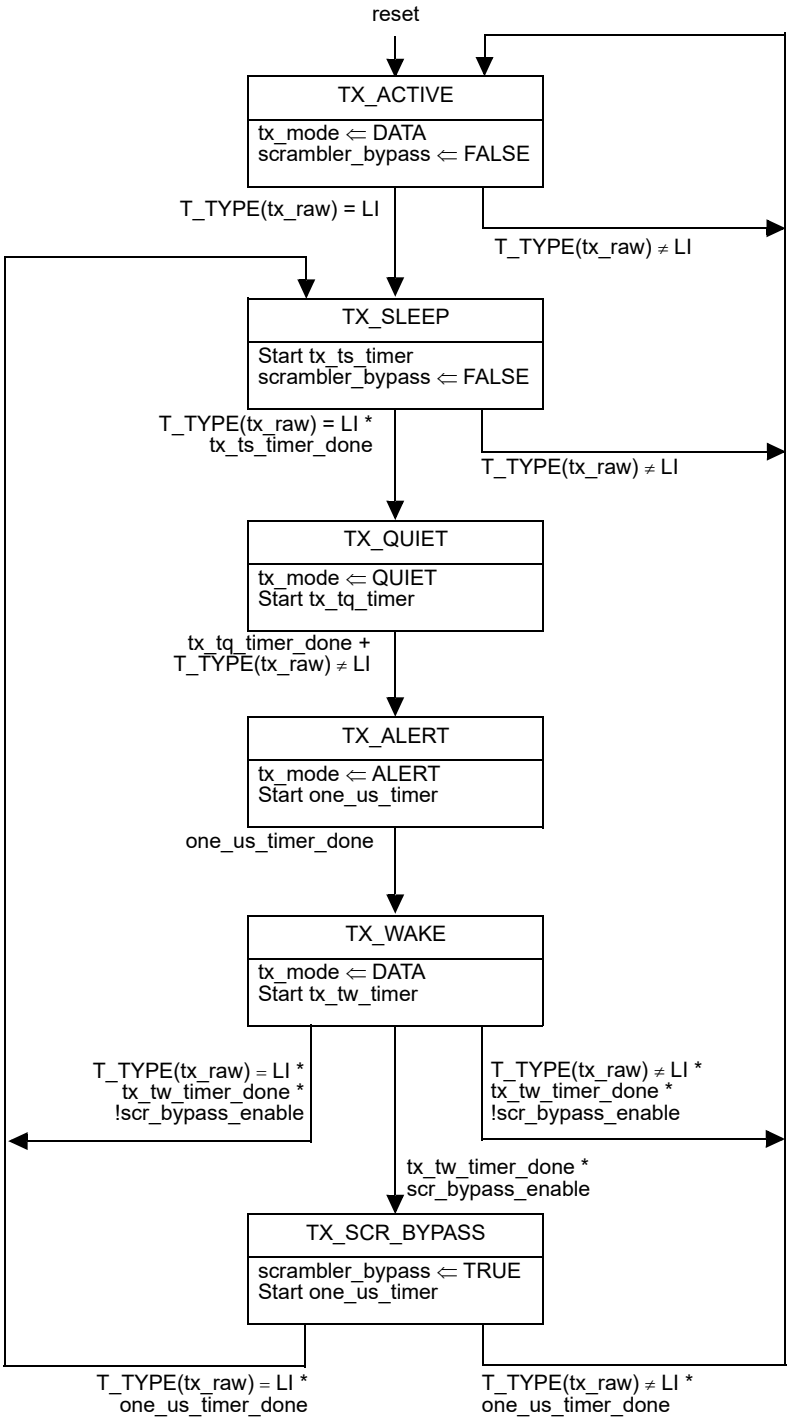


Figure 49–12—LPI Transmit state diagram

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

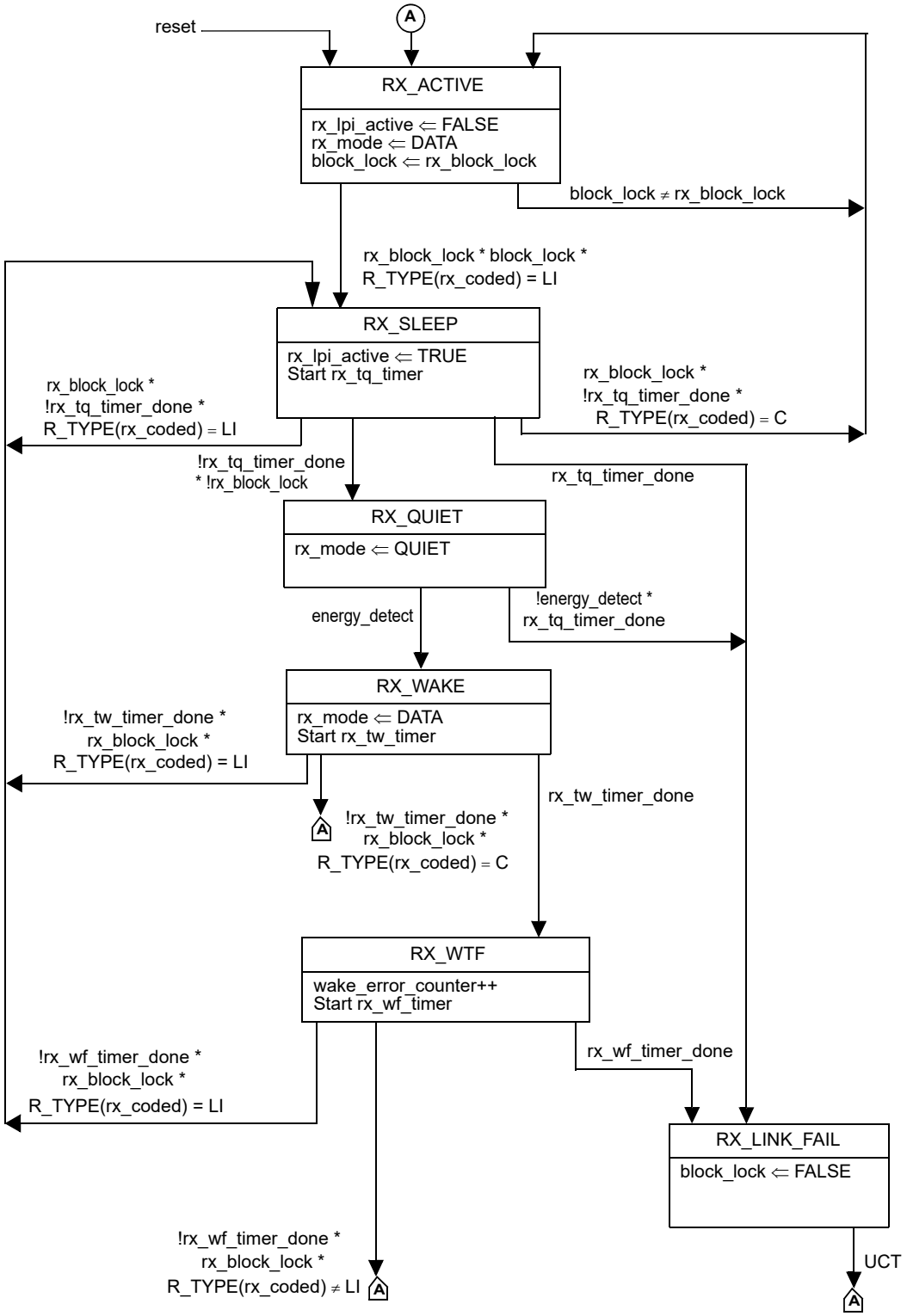


Figure 49–13—LPI Receive state diagram

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

Following a period of LPI, the receiver is required to achieve block synchronization within the wake-up time specified (see Figure 49–13). The implementation of the block synchronization state diagram should use techniques to ensure that block lock is achieved with minimal numbers of slip attempts. When the Clause 74 FEC is enabled, the receiver may use the knowledge that the link partner's transmitter will bypass the scrambler as part of the wake sequence. The idle sequence following this event will form a fixed pattern for the duration of the wake period.

The LPI functions shall use timer values for these state diagrams as shown in Table 49–2 for transmit and Table 49–3 for receive.

Table 49–2—Transmitter LPI timing parameters

Parameter	Description	Min	Max	Units
T_{SL}	Local Sleep Time from entering the TX_SLEEP state to when tx_mode is set to QUIET	4.9	5.1	µs
T_{QL}	Local Quiet Time from when tx_mode is set to QUIET to entry into the TX_ALERT state	1.7	1.8	ms
T_{WL}	Time spent in the TX_WAKE state	10.9	11.1	µs
T_{1U}	Time spent in the TX_ALERT and TX_SCR_BYPASS states	1.1	1.3	µs

Table 49–3—Receiver LPI timing parameters

Parameter	Description	Min	Max	Units
T_{QR}	The time the receiver waits for energy_detect to be set to TRUE while in the RX_SLEEP and RX_QUIET states before asserting receive fault	2	3	ms
T_{WR}	Time the receiver waits in the RX_WAKE state before indicating a wake time fault (when scr_bypass_enable = FALSE)	—	11.5	µs
T_{WR}	Time the receiver waits in the RX_WAKE state before indicating a wake time fault (when scr_bypass_enable = TRUE)	—	13.7	µs
T_{WTF}	Wake time fault recovery time	—	10	ms

49.2.14 PCS Management

The following objects apply to PCS management. If an MDIO Interface is provided (see Clause 45), they are accessed via that interface. If not, it is recommended that an equivalent access be provided.

49.2.14.1 Status

PCS_status:
: Indicates whether the PCS is in a fully operational state. It is only true if block_lock is true and hi_ber is false. This status is reflected in MDIO register 3.32.12. A latch low view of this status is reflected in MDIO register 3.1.2 and a latch high of the inverse of this status, Receive fault, is reflected in MDIO register 3.8.10.

block_lock:
: Indicates the state of the block_lock variable. This status is reflected in MDIO register 3.32.0. A latch low view of this status is reflected in MDIO register 3.33.15.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

hi_ber:
: Indicates the state of the hi_ber variable. This status is reflected in MDIO register 3.32.1. A latch high view of this status is reflected in MDIO register 3.33.14.

Rx LPI indication:
: For EEE capability, this variable indicates the current state of the receive LPI function. This flag is set to TRUE (register bit set to one) when the LPI receive state diagram is in any state other than RX_ACTIVE. This status is reflected in MDIO register 3.1.8. A latch high view of this status is reflected in MDIO register 3.1.10 (Rx LPI received).

Tx LPI indication:
: For EEE capability, this variable indicates the current state of the transmit LPI function. This flag is set to TRUE (register bit set to one) when the LPI transmit state diagram is in any state other than TX_ACTIVE. This status is reflected in MDIO register 3.1.9. A latch high view of this status is reflected in MDIO register 3.1.11 (Tx LPI received).

49.2.14.2 Counters

The following counters are reset to zero upon read and upon reset of the PCS. When they reach all ones, they stop counting. Their purpose is to help monitor the quality of the link.

ber_count:
: 6-bit counter that counts each time BER_BAD_SH state is entered. This counter is reflected in MDIO register bits 3.33.13:8. Note that this counter counts a maximum of 16 counts per 125 μs since the BER_BAD_SH can be entered a maximum of 16 times per 125 μs window.

errored_block_count:
: 8-bit counter. When the receiver is in normal mode, errored_block_count counts once for each time RX_E state is entered. This counter is reflected in MDIO register bits 3.33.7:0.

test_pattern_error_count:
: 16-bit counter. When the receiver is in test-pattern mode, the test_pattern_error_count counts errors as described in 49.2.12. This counter is reflected in MDIO register bits 3.43.15:0.

49.2.14.3 Test mode control

tx_test_mode:
: Boolean variable controlling transmit channel operating mode. When false, the transmit channel operates in normal mode. When true, the transmit channel operates in test-pattern mode.

rx_test_mode:
: Boolean variable controlling receive channel operating mode. When false, the receive channel operates in normal mode. When true, the receive channel operates in test-pattern mode.

49.2.14.4 Loopback

The PCS shall be placed in Loopback mode when the Loopback bit in MDIO register 3.0.14 is set to a logic one. In this mode, the PCS shall accept data on the transmit path from the XGMII and return it on the receive path to the XGMII. In addition, the PCS shall transmit a continuous stream of 0x00FF data words to the PMA or WIS sublayer, and shall ignore all data presented to it by the PMA or WIS sublayer.

49.2.15 Delay constraints

Predictable operation of the MAC Control PAUSE operation (Clause 31, Annex 31B) demands that there be an upper bound on the propagation delays through the network. This implies that MAC, MAC Control sublayer, and PHY implementations conform to certain delay maxima, and that network planners and

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

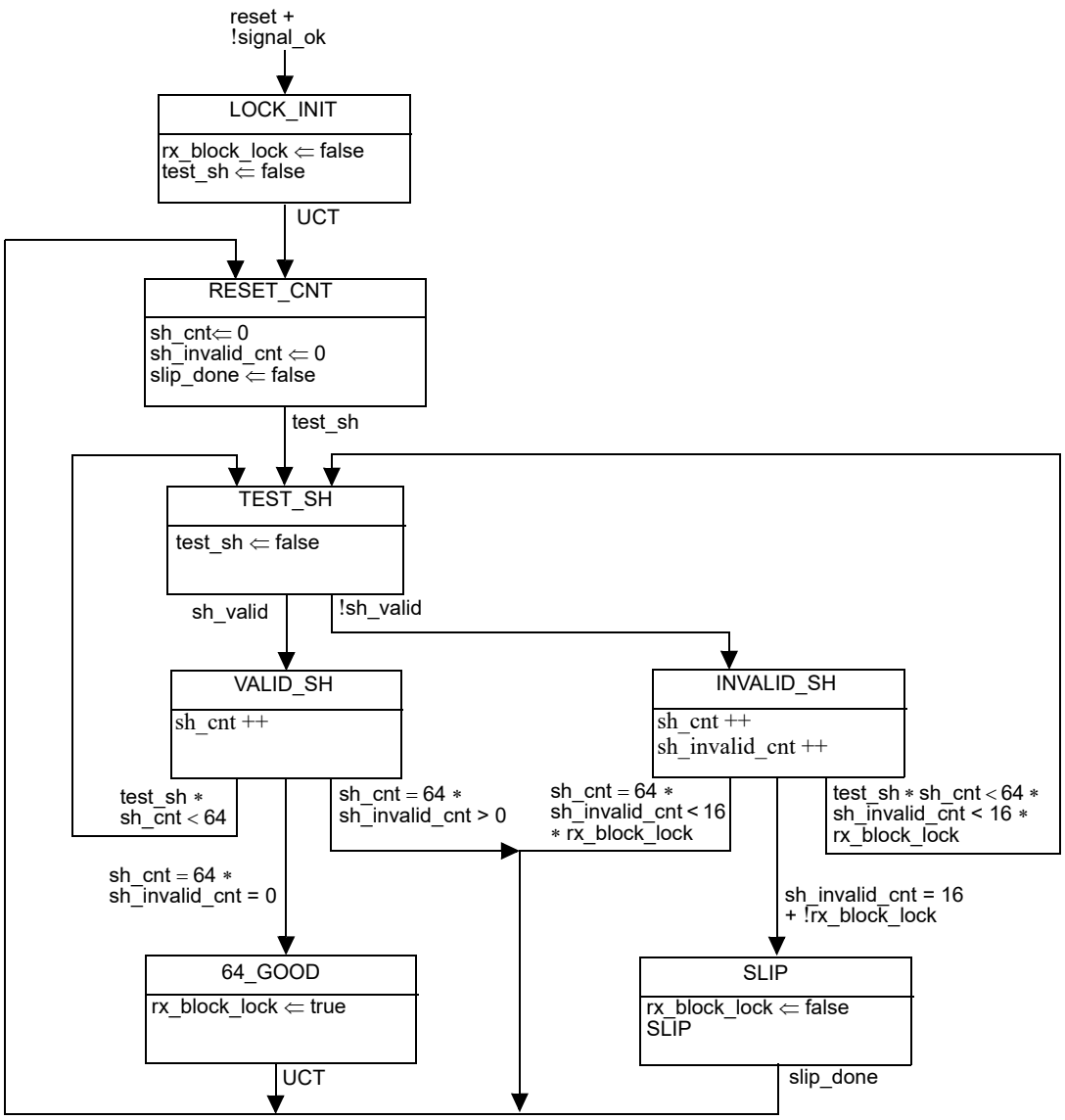


Figure 49–14—Lock state diagram

administrators conform to constraints regarding the cable topology and concatenation of devices. The sum of transmit and receive delay contributed by the 10GBASE-R PCS shall be no more than 3584 BT.

49.2.16 Auto-Negotiation for Backplane Ethernet

The following requirements apply to a PCS used with a 10GBASE-KR PMD. Support for the Auto-Negotiation process defined in Clause 73 is mandatory. The PCS shall support the primitive AN_LINK.indication(link_status) (see 73.9). The parameter link_status shall take the value FAIL when PCS_status=false and the value OK when PCS_status=true. The primitive shall be generated when the value of link_status changes.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

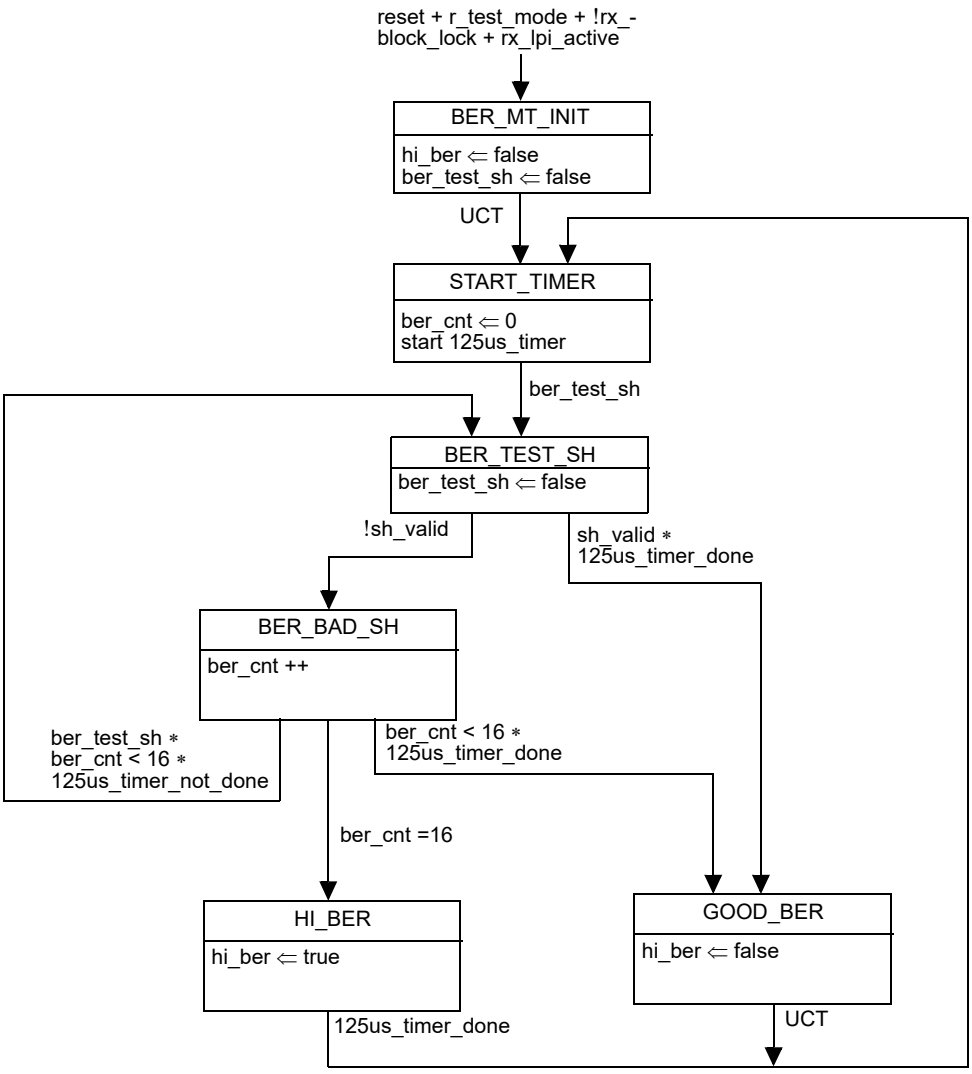


Figure 49–15—BER monitor state diagram

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

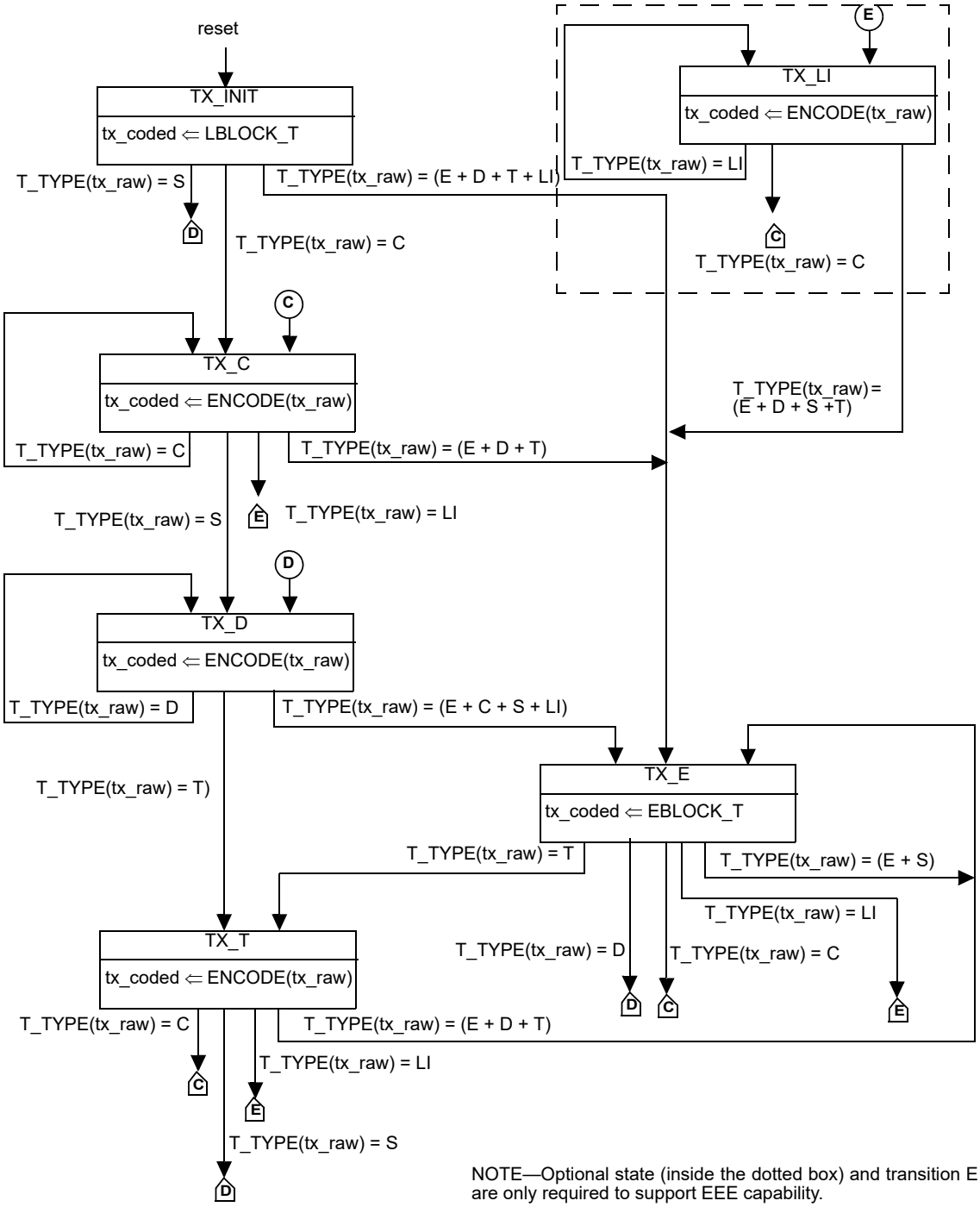


Figure 49–16—Transmit state diagram

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

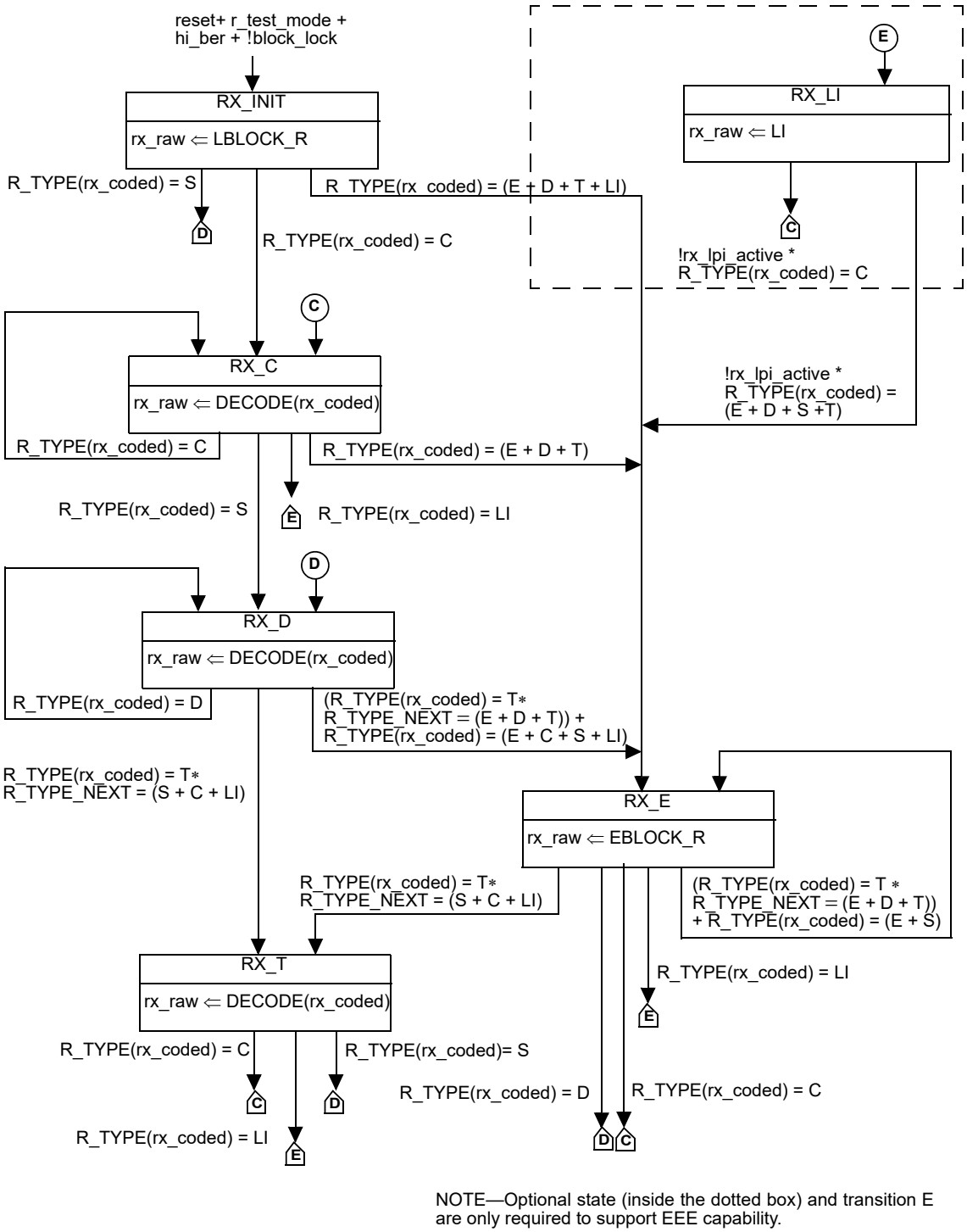


Figure 49–17—Receive state diagram

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.3 Protocol implementation conformance statement (PICS) proforma for Clause 49, Physical Coding Sublayer (PCS) type 10GBASE-R[96]

49.3.1 Introduction

The supplier of a protocol implementation that is claimed to conform to Clause 49, Physical Coding Sublayer (PCS), type 10GBASE-R, shall complete the following protocol implementation conformance statement (PICS) proforma. A detailed description of the symbols used in the PICS proforma, along with instructions for completing the PICS proforma, can be found in Clause 21.

49.3.2 Identification

49.3.2.1 Implementation identification

Supplier[1]	
Contact point for inquiries about the PICS[1]	
Implementation Name(s) and Version(s)[1,3]	
Other information necessary for full identification—e.g., name(s) and version(s) for machines and/or operating systems; System Name(s)[2]	
NOTE 1—Required for all implementations. NOTE 2—May be completed as appropriate in meeting the requirements for the identification. NOTE 3—The terms Name and Version should be interpreted appropriately to correspond with a supplier's terminology (e.g., Type, Series, Model).	

49.3.2.2 Protocol summary

Identification of protocol standard	IEEE Std 802.3-2022, Clause 49, Physical Coding Sublayer (PCS), type 10GBASE-R
Identification of amendments and corrigenda to this PICS proforma that have been completed as part of this PICS	
Have any Exception items been required? No [ ] Yes [ ] (See Clause 21; the answer Yes means that the implementation does not conform to IEEE Std 802.3-2022.)	
Date of Statement	

[96] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this subclause so that it can be used for its intended purpose and may further publish the completed PICS.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.3.3 Major capabilities/options

Item	Feature	Subclause	Value/Comment	Status	Support
XSBI	XBSI compatibility interface	51, 49.1.5	Compatibility interface is supported	O	Yes [ ] No [ ]
XGE	XGMII compatibility interface	46, 49.1.5	Compatibility interface is supported	O	Yes [ ] No [ ]
*MD	MDIO	45, 49.2.14	Registers and interface supported	O	Yes [ ] No [ ]
*WIS	Supports operation with a WIS	49.1.5		O.1	Yes [ ] No [ ]
*PMA	Supports operation directly connected to a PMA	49.1.5		O.1	Yes [ ] No [ ]
*JTM	Supports test-pattern mode	49.2.2		PMA:M	Yes [ ] N/A[ ]
*LPI	Implementation of LPI	49.2.4.4		O	Yes [ ] No [ ]

49.3.4 PICS Proforma Tables for PCS, type 10GBASE-R

49.3.4.1 Coding rules

Item	Feature	Subclause	Value/Comment	Status	Support
C1	Encoder implements the code as specified	49.2.13.2.3 and 49.2.4		M	Yes [ ]
C2	Decoder implements the code as specified	49.2.13.2.3 49.2.4		M	Yes [ ]
C3	IDLE control code insertion and deletion	49.2.4.7	Insertion or Deletion in groups of 4 /I/s	M	Yes [ ]
C4	IDLE control code deletion	49.2.4.7	When deleting /I/s, the first four characters after a /T/ shall not be deleted.	M	Yes [ ]
C5	Sequence ordered set deletion	49.2.4.10	Only one whole ordered set of two consecutive sequence ordered sets may be deleted	M	Yes [ ]

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.3.4.2 Scrambler and Descrambler

Item	Feature	Subclause	Value/Comment	Status	Support
S1	Scrambler	49.2.6	Performs as shown in Figure 49–8	M	Yes [ ]
S2	Descrambler	49.2.10	Performs as shown in Figure 49–10	M	Yes [ ]

49.3.5 Test-pattern modes

Item	Feature	Subclause	Value/Comment	Status	Support
JT1	Square wave and pseudo-random transmit test-pattern generators are implemented	49.2.8	Performs as in 49.2.8	JTM:M	Yes [ ] N/A[ ]
JT2	Pseudo-random receive test-pattern checker is implemented	49.2.12	Performs as in 49.2.12	JTM:M	Yes [ ] N/A[ ]
JT3	Transmit and receive test-pattern modes can operate simultaneously	49.2.2		JTM:M	Yes [ ] N/A[ ]
JT4	Reject transmit test-pattern mode when WIS is attached	49.2.8		JTM:O	Yes [ ] No [ ] N/A[ ]
JT5	Reject receive test-pattern mode when WIS is attached	49.2.12		JTM:O	Yes [ ] No [ ] N/A[ ]
*JT6	Support for PRBS31 test pattern	49.2.8		JTM:O	Yes [ ] No [ ] N/A[ ]
JT7	PRBS31 test-pattern generator is implemented	49.2.8	Performs as in 49.2.8	JT6:M	Yes [ ] N/A[ ]
JT8	PRBS31 test-pattern checker is implemented	49.2.12	Performs as in 49.2.12	JT6:M	Yes [ ] N/A[ ]
*JT9	Support for PRBS9 transmit test pattern	49.2.8		JTM:O	Yes [ ] No [ ] N/A[ ]
JT10	PRBS9 transmit test pattern is implemented	49.2.8	Performs as in 49.2.8	JT9:M	Yes [ ] N/A[ ]

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.3.5.1 Bit order

Item	Feature	Subclause	Value/Comment	Status	Support
B1	Transmit bit order	49.2.7	Placement of bits into tx_data-group<15:0> as specified in 49.2.7	M	Yes [ ]
B2	Receive bit order	49.2.9	Placement of bits from rx_data-group<15:0> into blocks as specified in 49.2.9	M	Yes [ ]

49.3.6 Management

Item	Feature	Subclause	Value/Comment	Status	Support
M1	Equivalent access to PCS Management objects is provided	49.2.14		!MD:O	Yes [ ] No [ ] N/A [ ]

49.3.6.1 State diagrams

Item	Feature	Subclause	Value/Comment	Status	Support
SM1	Lock	49.2.13.3	Meets the requirements of Figure 49–14	M	Yes [ ]
SM2	BER Monitor	49.2.13.3	Meets the requirements of Figure 49–15	M	Yes [ ]
SM3	Transmit	49.2.13.3	Meets the requirements of Figure 49–16	M	Yes [ ]
SM4	Receive	49.2.13.3	Meets the requirements of Figure 49–17	M	Yes [ ]
SM5	PCS_R_Status	49.2.2	PCS_R_STATUS is FAIL in RX_INIT state; OK otherwise	M	Yes [ ]

49.3.6.2 WIS

Item	Feature	Subclause	Value/Comment	Status	Support
W1	Supports WIS_SIGNAL.request	49.2.2	PCS_R_STATUS is FAIL when in RX_INIT state	WIS:M	Yes [ ] N/A[ ]

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.3.6.3 Loopback

Item	Feature	Subclause	Value/Comment	Status	Support
L1	Supports Loopback	49.2.14.4	Performs as in 49.2.14.4	M	Yes [ ]

49.3.6.4 Delay Constraints

Item	Feature	Subclause	Value/Comment	Status	Support
TIM1	PCS Delay Constraint	49.2.15	No more than 3584 BT for sum of transmit and receive path delays	M	Yes [ ]

49.3.6.5 Auto-Negotiation for Backplane Ethernet functions

Item	Feature	Subclause	Value/Comment	Status	Support
*AN1	Support for use with a 10GBASE-KR PMD	49.2.16	AN technology dependent interface described in Clause 73	O	Yes [ ] No [ ]
AN2	AN_LINK.indication primitive	49.2.16	Support of the primitive AN_LINK.indication(link_status), when the PCS is used with 10GBASE-KR PMD	AN1:M	Yes [ ] N/A [ ]
AN3	link_status parameter	49.2.16	Takes the value OK or FAIL, as described in 49.2.16	AN1:M	Yes [ ] N/A [ ]
AN4	Generation of AN_LINK.indication primitive	49.2.16	Generated when the value of link_status changes	AN1:M	Yes [ ] N/A [ ]

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.

49.3.6.6 LPI functions

Item	Feature	Subclause	Value/Comment	Status	Support
LP-01	Insertion and deletion of LPIs in groups of four	49.2.4.7		LPI:M	Yes [ ] N/A [ ]
LP-02	Unscrambled data transmitted when scrambler_bypass = TRUE	49.2.6		LPI:M	Yes [ ] N/A [ ]
LP-03	Scrambler continues to operate as normal when scrambler_bypass = TRUE	49.2.6		LPI:M	Yes [ ] N/A [ ]
LP-04	scr_bypass_enable = TRUE when FEC is in use	49.2.13.2.2		LPI:M	Yes [ ] N/A [ ]
LP-05	Transmit state diagrams	49.2.13.3	Support additions to Figure 49–16 for EEE capability	LPI:M	Yes [ ] N/A [ ]
LP-06	Receive state diagrams	49.2.13.3	Support additions to Figure 49–17 for EEE capability	LPI:M	Yes [ ] N/A [ ]
LP-07	LPI transmit state diagrams	49.2.13.3.1	Meets the requirements of Figure 49–12	LPI:M	Yes [ ] N/A [ ]
LP-08	LPI receive state diagrams	49.2.13.3.1	Meets the requirements of Figure 49–13	LPI:M	Yes [ ] N/A [ ]
LP-09	LPI transmit timing	49.2.13.3.1	Meets the requirements of Table 49–2	LPI:M	Yes [ ] N/A [ ]
LP-10	LPI receive timing	49.2.13.3.1	Meets the requirements of Table 49–3	LPI:M	Yes [ ] N/A [ ]

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: MINCYT. Downloaded on April 09,2024 at 15:31:55 UTC from IEEE Xplore. Restrictions apply.